

ExamSentinel — 30-Part Windsurf Build Series

Tech stack (locked): Python 3.11 · Flask + SQLAlchemy · PostgreSQL on Railway · Tkinter · requests · pywin32 · wmi · ctypes · psutil · PyInstaller. **Audience:** Claude Opus 4.6/4.7 inside Windsurf as the implementing coder. You (the human) paste each prompt in order, run the verification block before moving to the next, and ship.

How to Use This Document

1. Work top-to-bottom. Never start Part N until every checkbox in **Verification of Part N-1** is green.
 2. Each "Prompt for Windsurf" block is the literal text you paste into Windsurf's chat. Don't paraphrase it.
 3. When a prompt names files or modules, those names are load-bearing — later prompts reference them.
 4. Run the server locally (Parts 5-6) before connecting to Railway (Part 7). Don't try to debug deploy issues and code issues at the same time.
 5. If Windsurf produces something that diverges from the prompt's contract, push back in chat and ask for the corrected version before accepting.
-

Architectural Overview (Distilled from Parts 1-4)

System topology. ExamSentinel is two binaries plus one cloud service. The **server** is a Flask + SQLAlchemy REST API hosted on Railway with a Railway-managed PostgreSQL instance attached. The **client** is a single Windows .exe (built with PyInstaller from a Tkinter project) that runs on examiner / student / teacher Windows 10/11 machines and talks to the server over HTTPS. There is no peer-to-peer communication between clients; everything goes through the server.

Domain model. A `User` carries identity and credentials. `Student` and `Teacher` extend `User` with role-specific fields (roll number, employee code). A `Teacher` owns `Course`s. A `Course` enrolls `Student`s through an `Enrollment` join entity. A `Course` contains `Exam`s. An `Exam` is composed of `Question`s. When a `Student` sits an `Exam`, an `ExamSession` is created — a single attempt by a single student at a single exam. The session passes through three phases: pre-session (VM gating), live (lockdown active, answers being saved), post (submission and final log shipping). `Answer`s belong to a session-question pair. `IncidentLog`s belong to a session and record everything notable: VM detection results, lockdown violations, focus losses, blacklist process kills, clipboard scrubs, and timing/thermal forensic samples for stealth-VM evidence.

Exam session lifecycle (sequence).

1. Student logs in. Client receives JWT with role.
2. Student picks an active exam from their course list.
3. Client creates an `ExamSession` on the server with status `pre_check`.
4. Client runs Standard VM Detection. If positive, ship incident, abort session as `aborted_vm`, refuse to proceed. Re-running the .exe re-runs the check.
5. Client runs Stealth VM Detection (CPUID, RDTSC timing, evasion heuristics). If positive, same outcome with type `aborted_stealth_vm`.
6. Lockdown engages: keyboard hook, process killer, clipboard scrubber, fullscreen takeover, taskbar hide, focus monitor, mouse boundary lock, multi-monitor block.
7. Server transitions session to `in_progress`. Client polls server time-remaining; user answers; each answer is auto-saved.
8. On submit (manual or auto on timeout): lockdown disengages cleanly, answers are finalized, score is computed for MCQs.
9. All queued incidents are flushed to the server. Session is marked `submitted`.
10. Teacher reviews the session: answers, computed score, manual grading for short-answer questions, full incident timeline.

Threat model (summary). The student is the adversary. Attacks considered: app switching, copy/paste, screenshots, screen recorders, helper apps (Discord, AnyDesk, TeamViewer), inspecting answer JSON via clipboard, running the exam inside a VM with hidden tooling, alt-launching restricted apps via Win/Ctrl+Esc/Task Manager, multi-monitor cheat displays. Out of scope: a second physical device (phone), a second human in the room, OS-level rootkits, hardware modifications. Defense layers: pre-exam VM gates → live OS lockdown → continuous monitoring → forensic logging → teacher review. No claim of bypass-proofness; the system raises the cost of cheating and produces a defensible evidence trail.

Repository structure.

```
ExamSentinel/
docs/           # ARCHITECTURE.md, API.md, THREAT_MODEL.md, DEMO_DAY.md
server/        # Flask backend (deploys to Railway)
  app/
    models/     # SQLAlchemy entities
    routes/     # Blueprint per resource
    services/   # Business logic (auth, grading, sessions)
    utils/      # JWT helpers, validators, errors
  migrations/   # Alembic
  seed/         # Dummy-data generator
  Procfile, runtime.txt, requirements.txt, .env.example
client/        # Tkinter desktop app (packaged via PyInstaller)
  app/
    screens/    # One module per screen
    services/   # API client wrapper, session state
    lockdown/   # Keyboard, process, clipboard, fullscreen, focus,
monitor
  vm_detect/    # Standard + stealth VM detection
  ui/          # Theme constants, reusable widgets
  assets/      # Icon, manifest
  build/       # PyInstaller spec, build.bat
  requirements.txt
  README.md
```

Part 1 — Architectural Design Document

Stage: Design Goal: Produce `docs/ARCHITECTURE.md` capturing the full system design before any code is written. **Prerequisites:** None (foundation).

Verification of Part 0 — Environment Preflight. Run these checks on your Windows 11 machine before starting Part 1.

- ☐ Install Python 3.11 from python.org (check "Add to PATH" during install). In a fresh terminal `python --version` returns `Python 3.11.x`. If your machine had an older Python, prefer the `py -3.11` launcher.
- ☐ `pip --version` returns a pip tied to Python 3.11.
- ☐ `git --version` returns 2.x. Configure `git config --global user.name` and `user.email` if not already set.
- ☐ You can sign into github.com in a browser. Create an empty private repository named `ExamSentinel` (do NOT initialize with README or .gitignore — we'll push from local).
- ☐ You can sign into railway.app in a browser. Hobby plan (\$5/month) is active on your account. You don't need the Railway CLI for this series — all Railway interactions are via the web dashboard and via Railway's GitHub auto-deploy.

☐ `pip install pyinstaller` succeeds in your default Python 3.11 environment. We won't use it until Part 28, but verifying installability now catches problems early.

☐ Windsurf opens. In the model selector you can see Claude Opus 4.7 (or 4.6) and Claude Sonnet 4.5 (or newer). Make a fresh empty workspace folder named `ExamSentinel/` and open it in Windsurf.

☐ In a terminal inside that folder run `git init` and make an initial empty commit (`git commit --allow-empty -m "init"`), then add the GitHub remote (`git remote add origin <url>`) and `git push -u origin main`.

Recommended model: Claude Opus 4.7 (alternative: Claude Sonnet 4.5; fallback: GPT-5). Use Opus here — design quality compounds across the series.

Prompt for Windsurf:

You are joining the ExamSentinel project at the architecture stage. The repository is empty except for a git history. Produce a single markdown file at `docs/ARCHITECTURE.md` that fully describes the system before any code is written. Cover, with headings: system topology and the rationale for a Flask backend on Railway with PostgreSQL plus a Tkinter Windows client; the complete domain model (User, Student, Teacher, Course, Enrollment, Exam, Question, ExamSession, Answer, IncidentLog) including each entity's purpose, key attributes in prose, primary relationships, and lifecycle states; an entity-relationship description in prose plus a Mermaid ER diagram block; module boundaries for both server and client matching the folder layout I will describe below; a sequence description (numbered prose, plus a Mermaid sequence diagram block) for one full exam session from student login through submission, explicitly including the pre-session VM gates and the live lockdown phase; a deployment topology section showing how the client reaches Railway over HTTPS; and a rationale section justifying technology choices (why Flask, why SQLAlchemy, why Tkinter, why JWT, why PyInstaller single-file, why PostgreSQL).

The folder layout to document is exactly: top-level `docs/`, `server/` (with `app/models/`, `app/routes/`, `app/services/`, `app/utils/`, `migrations/`, `seed/`), `client/` (with `app/screens/`, `app/services/`, `app/lockdown/`, `app/vm_detect/`, `app/ui/`, `assets/`, `build/`), and a top-level `README.md`. Document this layout but do not yet create the folders or files — that is Part 4's job.

The exam session lifecycle has three phases in this order: pre-session (Standard VM detection then Stealth VM detection — both must pass), live (Windows lockdown engaged, student answers questions, answers auto-save), post (submission, lockdown disengage, log flush, teacher review). Make this ordering explicit and call out that re-running the .exe always re-runs both VM gates from scratch.

Produce the document in clear, concise engineering prose. Use Mermaid for diagrams. No code samples. Length target: enough to fully specify the system, no padding. After writing, commit the file with message "docs: architectural overview".

Definition of Done: `docs/ARCHITECTURE.md` exists, is committed, contains all the sections listed above with two Mermaid diagrams (ER and sequence), and a teammate could read it and understand the system without asking clarifying questions.

Part 2 — REST API Contract & Auth Flow

Stage: Design Goal: Produce `docs/API.md` defining every backend endpoint, request/response shapes (in prose), auth flow, and error conventions, locked before any handler is written.

Prerequisites: Part 1.

Verification of Part 1.

- ☐ `docs/ARCHITECTURE.md` exists at the repository root path.
- ☐ It contains both an ER diagram (Mermaid `erDiagram`) and a sequence diagram (Mermaid `sequenceDiagram`).
- ☐ Every entity from the domain model is present with attributes and relationships described.
- ☐ The exam session lifecycle is described with the three phases in the locked order: pre-session VM gates → live lockdown → post submission.
- ☐ The folder structure section names every folder used in subsequent parts.
- ☐ The file is committed and pushed to GitHub.

Recommended model: Claude Opus 4.7 (alternative: Claude Sonnet 4.5).

Prompt for Windsurf:

Read `docs/ARCHITECTURE.md`. Produce `docs/API.md` defining the complete REST API contract. The API is versioned under the prefix `/api/v1`. Authentication is JWT bearer tokens issued at login; tokens carry the user id and role and expire after twelve hours. Document the auth flow end-to-end: registration with role selection (student or teacher) and role-specific fields, login returning a token plus user profile, a "me" endpoint for clients to refresh state, password hashing with werkzeug, and the standard 401 / 403 / 422 / 500 error envelopes.

For every endpoint specify: HTTP method, path, role(s) required, what the request body contains in prose (no JSON code blocks — describe the fields and their types in sentences), what the response contains in prose, success status code, and the validation rules. Group endpoints under sections: Auth, Departments and Users, Courses and Enrollments, Exams and Questions, Sessions and Answers (the student-facing exam flow), Incident Logs (the lockdown/VM event ingestion), Teacher Reporting. Cover at minimum: register, login, current user, list departments, list students with filters, course CRUD for teachers, enroll student by email, list courses for current user, exam CRUD with nested questions, activate/deactivate exam, list active exams for student, start session (returns session id, server-authoritative deadline, and the question set without correct answers), save individual answer (upsert keyed on session+question), submit session (auto-grades MCQs, leaves text questions for manual grading), get time remaining (server-authoritative), get session result, ingest a single incident log (including optional thermal and timing fields for stealth-VM evidence), list sessions for an exam, get full session detail with answers and incidents for teacher review, manually grade a text answer, and exam analytics summary.

Define an error envelope convention in prose: every non-2xx response has a stable shape with an error code, a human message, and optional field-level details. Define pagination conventions for list endpoints (page and page_size query params, default 20, max 100). Define the rule that the correct answer of an MCQ is never returned to a student-role caller, only to teacher-role callers.

Commit as "docs: REST API contract and auth flow". No code, no executable schema files — this is a contract document.

Definition of Done: `docs/API.md` is complete, committed, and every later backend prompt can reference it as the source of truth for endpoint shapes.

Part 3 — Threat Model & Security Design

Stage: Design Goal: Produce `docs/THREAT_MODEL.md` and `docs/SECURITY_DESIGN.md` specifying what the security engine defends against, what it does not, and the precise mechanism for each defense. **Prerequisites:** Parts 1–2.

Verification of Part 2.

- ☐ `docs/API.md` exists, committed.
- ☐ Every endpoint listed in the architecture overview's API section is present in `API.md`.
- ☐ The auth flow section specifies JWT with role claim and twelve-hour expiry.
- ☐ The MCQ correct-answer hiding rule for student-role callers is explicit.
- ☐ An error envelope convention is documented.
- ☐ The contract distinguishes student-callable endpoints from teacher-callable endpoints with explicit role tags on each.

Recommended model: Claude Opus 4.7 (alternative: Claude Sonnet 4.5). Opus pays off here — security writing benefits from careful reasoning.

Prompt for Windsurf:

Read `docs/ARCHITECTURE.md` and `docs/API.md`. Produce two documents.

First, `docs/THREAT_MODEL.md`: identify the adversary as the student taking the exam. Enumerate in-scope threats with one paragraph each: app switching to consult notes, copy-paste of question text or answer text to/from other apps, full-screen screenshots and screen recorders, helper apps (Discord, AnyDesk, TeamViewer, OBS, Snipping Tool), launching the exam inside a VirtualBox or VMware or Hyper-V guest with the host serving as a cheat surface, hardened/stealth VMs with guest tools removed and identifiers spoofed, alt-launch via Win key / Ctrl+Esc / Ctrl+Shift+Esc / Task Manager / Run dialog, multi-monitor setups with answers on the second display, network exfiltration of question content. Enumerate explicitly out-of-scope threats: a second physical device such as a phone, another human in the room, OS-level rootkits or signed-driver malware, hardware-level attacks. Conclude with a one-paragraph statement of defense philosophy: defense in depth raising the cost of cheating, plus a defensible evidence trail for the teacher.

Second, `docs/SECURITY_DESIGN.md`: for every defense, describe what it does, the Windows mechanism it uses (named in prose only, no code), how it integrates with the lockdown manager, what events it logs, and known limitations. Cover: pre-exam Standard VM Detection (WMI hardware fingerprints across BIOS / system manufacturer / motherboard / video controller, MAC address OUI checks for VMware / VirtualBox / Hyper-V ranges, registry artifact checks for guest tools, running-process checks for `vmtoolsd` / `VBoxService` / `vmwaretray`); pre-exam Stealth VM Detection (CPUID hypervisor-present bit at leaf 1 ECX bit 31, CPUID leaf 0x40000000 hypervisor vendor string read, RDTSC timing deltas across instructions known to trap on hypervisors, sleep-versus-wallclock skew, suspicious-driver enumeration, deep registry artifacts that survive guest-tool removal, optional thermal-zone constancy via WMI as a soft signal); the rule that both VM gates must pass before the live phase, that any positive detection logs an incident and refuses to proceed, and that re-running the .exe re-runs the gates; live-phase Windows Lockdown (low-level keyboard hook blocking Alt+Tab / Alt+F4 / Win key both sides / Ctrl+Esc / Alt+Esc / Ctrl+Shift+Esc / Print Screen, but allowing typing keys; process scanner with a named blacklist that kills offending processes via `psutil`; clipboard scrubber clearing on a short interval; right-click suppression at the Tkinter level; fullscreen takeover via `pywin32` with `topmost` flag and titlebar removal; taskbar hide; focus monitor that re-foregrounds the exam window on focus loss and logs the event; mouse boundary lock via `ClipCursor`; multi-monitor detection that blocks exam start and warns on hot-plug); the violation logging pipeline (every defense emits structured events to a single in-memory queue; a background flusher posts to the incidents endpoint with retry and offline queuing; on submit, queue is drained before lockdown disengage); clean-shutdown contract (every hook, thread, cursor clip, fullscreen state, and taskbar tweak must be reversible — even on crash — via a finally-block teardown). State limitations honestly: a determined adversary can defeat any software-only lockdown; Print Screen suppression via keyboard hook does not stop hardware Print Screen scancodes on every keyboard; stealth VM detection is heuristic, not proof. Document the `SKIP_VM_CHECK` developer flag for local testing. Commit both files with message "docs: threat model and security design".

Definition of Done: Both documents exist and committed. The security design is detailed enough that Parts 19–27 can be executed by referencing it without further design discussion.

Part 4 — Repository Scaffold, Dependency Manifests, Environment Templates

Stage: Design **Goal:** Materialize the folder structure, create dependency manifests for both server and client, write `.env.example` templates, set up `.gitignore`, and commit. No application code yet. **Prerequisites:** Parts 1–3.

Verification of Part 3.

- ☐ `docs/THREAT_MODEL.md` and `docs/SECURITY_DESIGN.md` both exist.
- ☐ Threat model lists at least eight in-scope threats and three out-of-scope threats.

- ☐ Security design covers Standard VM detection mechanisms (WMI, MAC OUI, registry, processes), Stealth VM detection mechanisms (CPUID, RDTSC, vendor string, drivers, registry, thermal), and Lockdown mechanisms grouped as keyboard / process / clipboard / fullscreen / focus / mouse / multi-monitor.
- ☐ The clean-shutdown reversibility contract is explicit.
- ☐ The SKIP_VM_CHECK dev flag is documented.

Recommended model: Claude Sonnet 4.5 (alternative: Claude Opus 4.7). Sonnet is fine for scaffolding.

Prompt for Windsurf:

Create the full repository scaffold described in `docs/ARCHITECTURE.md`. Create empty Python package directories with `__init__.py` files for: `server/app`, `server/app/models`, `server/app/routes`, `server/app/services`, `server/app/utils`, `server/seed`, and `client/app`, `client/app/screens`, `client/app/services`, `client/app/lockdown`, `client/app/vm_detect`, `client/app/ui`. Also create empty (no-init) folders `server/migrations`, `client/assets`, `client/build`. Create a top-level `README.md` with one paragraph project description and a link to `docs/ARCHITECTURE.md`.

Write `server/requirements.txt` pinning these libraries to recent stable versions compatible with Python 3.11: flask, flask-sqlalchemy, flask-migrate, flask-jwt-extended, flask-cors, pycpg2-binary, sqlalchemy, alembic, python-dotenv, werkzeug, gunicorn, marshmallow, pytest. Use exact pins so Railway and local builds are reproducible.

Write `client/requirements.txt` pinning: requests, pywin32, wmi, psutil, screeninfo, python-dotenv, pyinstaller. Tkinter is not pinned because it ships with Python.

Write `server/.env.example` listing the env var names needed (no secrets — just names and a one-line comment per var): DATABASE_URL, SECRET_KEY, JWT_SECRET_KEY, FLASK_ENV, CORS_ALLOWED_ORIGINS, PORT. Write `client/.env.example` listing API_BASE_URL and SKIP_VM_CHECK with default values commented.

Write `server/Procfile` declaring the web process runs gunicorn against the Flask app factory module path (state the path in prose: the app module exposes a `create_app` factory; gunicorn invokes it). Write `server/runtime.txt` pinning Python 3.11.x.

Write a comprehensive `.gitignore` at the repository root covering Python (`__pycache__`, `*.pyc`, `.pytest_cache`, `.mypy_cache`), virtual environments under `.venv` and `venv`, build artifacts (`build/`, `dist/`, `*.egg-info`), env files (`.env`) everywhere but allow `.env.example`, IDE files (`.vscode/`, `.idea/`, `*.swp`), OS files (`Thumbs.db`, `.DS_Store`), and PyInstaller spec build outputs.

Make no other files. Commit as "scaffold: repository structure, dependency manifests, env templates, gitignore". Push.

Definition of Done: Running `git status` shows a clean tree on `main`. The folder structure exactly matches what the architecture document describes. Both requirements files exist with pinned versions. Both `.env.example` files exist. Procfile and runtime.txt exist. `.gitignore` covers Python, env, IDE, OS, and PyInstaller artifacts.

Part 5 — Flask App Skeleton, Health Endpoint, CORS, Env Config

Stage: Backend Goal: Stand up a minimal Flask app using the application factory pattern, with config loaded from environment, CORS configured, and a `/api/v1/health` endpoint returning a stable shape, runnable locally against a SQLite fallback database. **Prerequisites:** Part 4.

Verification of Part 4.

- ☐ All package directories exist with `__init__.py`.
- ☐ `server/requirements.txt` and `client/requirements.txt` are present and pinned.
- ☐ Both `.env.example` files exist with all variables named in Part 4.
- ☐ `server/Procfile` and `server/runtime.txt` exist.
- ☐ `.gitignore` is present and excludes `.env`, `__pycache__`, `dist/`, `build/`, `virtualenv` folders.
- ☐ `git status` is clean; the scaffold commit is pushed.

Recommended model: Claude Opus 4.7 (alternative: Claude Sonnet 4.5).

Prompt for Windsurf:

Read `docs/API.md`. Build the Flask backend skeleton inside `server/app/` using the application factory pattern. The factory function lives in `server/app/__init__.py` and returns a configured Flask instance. Configuration is loaded from a `Config` class in `server/app/config.py` which reads exclusively from environment variables — never hardcode credentials. The `Config` class reads `DATABASE_URL`, `SECRET_KEY`, `JWT_SECRET_KEY`, `FLASK_ENV`, `CORS_ALLOWED_ORIGINS` (comma-separated list), and `PORT`. If `DATABASE_URL` is missing, fall back to a local SQLite file at `server/dev.db` so local development works without Postgres installed; in any non-development `FLASK_ENV` the fallback is forbidden and missing `DATABASE_URL` must raise a clear startup error. Wire flask-cors with origins driven by `CORS_ALLOWED_ORIGINS` (default to `*` only when `FLASK_ENV` is development). Wire flask-sqlalchemy with the database URI from config but do not yet declare any models — that is Part 6. Initialize flask-migrate against the SQLAlchemy instance for future Alembic migrations.

Create a health blueprint at `server/app/routes/health.py` exposing `GET /api/v1/health`. The response includes a status string, the API version string `v1`, the resolved database dialect (sqlite or postgresql) without leaking the connection string, and the current UTC timestamp in ISO 8601. Register the blueprint inside the factory under the `/api/v1` prefix so the final path is `/api/v1/health`.

Create `server/wsgi.py` exposing an `app` object obtained by calling the factory — gunicorn on Railway will import this. Create `server/app.py` (or `server/run_dev.py` — choose one and stick with it) that runs the dev server via `flask --app` semantics, reading `PORT` from env with a default of 5000.

Add a top-level `Makefile` or batch helper inside `server/` named `dev.bat` (since the user is on Windows) that activates a venv, installs requirements, copies `.env.example` to `.env` if missing, and runs the dev server.

All requirements:

- Flask app
- CORS enabled
- environment variables (NOT hardcoded DB credentials)
- Railway-compatible deployment
- uses DATABASE_URL

Do not yet implement auth, models, or any other endpoint beyond health. Commit as "backend(skeleton): app factory, config, CORS, health endpoint".

Definition of Done: Running `dev.bat` (or the equivalent commands) starts the server on port 5000. Hitting `http://127.0.0.1:5000/api/v1/health` from a browser or curl returns 200 with a JSON body containing status, version, dialect, timestamp. Stopping the dev server and unsetting DATABASE_URL still works (SQLite fallback). With `FLASK_ENV=production` and DATABASE_URL unset, the server refuses to start and prints a clear error.

Part 6 — SQLAlchemy Domain Models

Stage: Backend **Goal:** Implement the full domain model as SQLAlchemy declarative classes matching `docs/ARCHITECTURE.md`, with proper relationships, indexes, and enum types — but no migrations applied yet. **Prerequisites:** Part 5.

Verification of Part 5.

- ☐ `python -m flask --app server.wsgi run --port 5000` (or your chosen entry) boots cleanly.
- ☐ `curl http://127.0.0.1:5000/api/v1/health` returns 200 with the documented body shape.
- ☐ With `FLASK_ENV=production` and DATABASE_URL unset, server boot fails fast with a clear message naming the missing variable.
- ☐ With FLASK_ENV unset, the SQLite fallback kicks in and `server/dev.db` is created on first connection.
- ☐ CORS preflight `curl -X OPTIONS -H "Origin: http://example.com" -H "Access-Control-Request-Method: GET" http://127.0.0.1:5000/api/v1/health` returns CORS headers in development mode.
- ☐ Commit is pushed.

Recommended model: Claude Opus 4.7 (alternative: Claude Sonnet 4.5).

Prompt for Windsurf:

Read `docs/ARCHITECTURE.md` and `docs/API.md`. Implement every domain entity as a SQLAlchemy declarative model under `server/app/models/`, one module per entity, plus a `server/app/models/__init__.py` that imports all models so SQLAlchemy's metadata is

populated. Use a shared `db` SQLAlchemy instance defined in `server/app/extensions.py` and refactor Part 5's factory to import it from there.

Entities and their key fields, in prose (you choose appropriate types and lengths):

User: surrogate primary key, full name, email (unique, indexed, case-insensitive lookup), password hash, role enum with values student and teacher, created_at, updated_at. Methods to set and verify password using `werkzeug.security`. A computed property for whether the user is a teacher or student.

Student: primary key that is also a foreign key to User (joined-table inheritance — Student extends User), roll number (unique, indexed), department reference (foreign key to Department), semester (small integer), is_eligible boolean default true. Relationship back to User one-to-one.

Teacher: primary key that is also a foreign key to User (joined-table inheritance), employee code (unique, indexed), designation. Relationship back to User one-to-one.

Department: surrogate primary key, name (unique), campus location, created_at.

Course: surrogate primary key, course code (unique, indexed), title, owner teacher (foreign key to Teacher), created_at. Relationship to enrollments and exams.

Enrollment: surrogate primary key, student foreign key, course foreign key, enrolled_at, with a composite unique constraint on (student, course).

Exam: surrogate primary key, course foreign key, title, duration in minutes, scheduled start time (nullable), total_marks (computed/cached), is_active boolean default false, created_at. Relationship to questions and sessions.

Question: surrogate primary key, exam foreign key, prompt text, type enum (mcq, short_answer), marks (integer), four MCQ option text fields (nullable for short_answer questions), correct_option enum (A/B/C/D, nullable for short_answer), order_index integer for question ordering inside an exam.

ExamSession: surrogate primary key, exam foreign key, student foreign key, status enum with values pre_check, in_progress, submitted, expired, aborted_vm, aborted_stealth_vm, started_at, ended_at (nullable), deadline_at (server-authoritative computed at start), score (nullable float), with an index on (student, exam).

Answer: surrogate primary key, session foreign key, question foreign key, answer text (nullable, free-form text or chosen option letter), marks_awarded (nullable float — null until graded), is_auto_graded boolean. Composite unique constraint on (session, question) to enforce one answer per question per session.

IncidentLog: surrogate primary key, session foreign key, occurred_at, type string with a controlled vocabulary (FOCUS_LOST, BLACKLISTED_PROCESS_KILLED, CLIPBOARD_SCRUB, KEYBOARD_BLOCKED, MULTI_MONITOR_DETECTED, VM_DETECTED, STEALTH_VM_DETECTED, FULLSCREEN_BREACH, MOUSE_ESCAPE, etc. — list the full vocabulary in a constants module under `server/app/utils/incident_types.py`), severity enum (info, warning, critical), description text, and three optional forensic columns: cpu_thermal_value (nullable float), timing_latency_ms (nullable float), evidence_path (nullable string). Indexed on (session, occurred_at).

Configure all relationships with appropriate back-populates and cascade rules: deleting an exam cascades to its questions and sessions; deleting a session cascades to its answers and

incident logs; users are never hard-deleted (do not configure cascade from User to Student/Teacher — those rows exist as long as the user does).

Use SQLAlchemy enum types where stated; use String columns with explicit lengths everywhere else (no unbounded VARCHAR). Add timestamp `(server_default)` for `created_at` fields. Use `(__repr__)` methods for debugging.

Backend requirements remain in force:

- Flask app
- CORS enabled
- environment variables (NOT hardcoded DB credentials)
- Railway-compatible deployment
- uses DATABASE_URL

Do not yet generate migrations or seed data. Do not yet implement routes for these entities. Commit as "backend(models): full domain entities and relationships".

Definition of Done: All model modules exist under `(server/app/models/)`. Importing the package from a Python REPL (`(python -c "from server.app.models import *")`) succeeds without error. The Flask app still boots and `(/api/v1/health)` still responds. No existing endpoints have regressed.

Part 7 — Railway Provisioning, Postgres, First Deploy, Migrations

Stage: Backend Goal: Deploy the skeleton + models to Railway with a managed Postgres instance, run the initial Alembic migration to materialize all tables on the live database, and verify the deployed health endpoint. **Prerequisites:** Parts 5-6.

Verification of Part 6.

- ☐ All entity modules exist under `(server/app/models/)`.
- ☐ `(python -c "from server.app import create_app; from server.app.extensions import db; app=create_app(); ctx=app.app_context(); ctx.push(); print(sorted(db.metadata.tables.keys()))")` lists every table: users, students, teachers, departments, courses, enrollments, exams, questions, exam_sessions, answers, incident_logs.
- ☐ All declared enums and unique constraints are present (inspect via `(db.metadata.tables['enrollments'].constraints)` and similar).
- ☐ `(/api/v1/health)` still returns 200 — no regressions.
- ☐ Commit pushed.

Recommended model: Claude Sonnet 4.5 (alternative: Claude Opus 4.7).

Prompt for Windsurf:

Two parallel tracks: deployment configuration and migration setup. Do them in this order.

Migration setup. Initialize Alembic via flask-migrate against the existing SQLAlchemy metadata. The migrations directory is `server/migrations` (already scaffolded as an empty folder). Generate the initial autogenerated migration that creates every table from Part 6. Inspect the generated script and verify it includes all tables, enums, indexes, and unique constraints — fix anything autogenerated missed (notably composite unique constraints sometimes need manual adjustment). Apply the migration to the local SQLite dev database to verify it runs cleanly, then roll back and re-apply to confirm idempotence at the migration script level.

Deployment configuration. Confirm `server/Procfile` declares `web: gunicorn --chdir server wsgi:app --bind 0.0.0.0:$PORT --workers 2` (adjust paths to match your actual factory location). Confirm `server/runtime.txt` pins Python 3.11.x. Add a `release` process to the Procfile that runs `flask --app server.wsgi db upgrade` — Railway will run this on every deploy to apply pending migrations automatically.

Now write a single markdown file at `docs/DEPLOY_RAILWAY.md` that walks the human through the Railway dashboard steps, since Railway provisioning is GUI-driven and not scriptable from this prompt. The document must walk through: creating a new Railway project from the GitHub repo (the human has already pushed); attaching a Postgres database via Railway's "+ New" → "Database" → "PostgreSQL" flow; observing that Railway auto-injects `DATABASE_URL` into the web service environment via reference variables (or instructing the human to add it manually as a reference variable if Railway didn't do it automatically); manually adding `SECRET_KEY`, `JWT_SECRET_KEY`, `FLASK_ENV=production`, `CORS_ALLOWED_ORIGINS=*` (we'll tighten this later), with values the human generates locally with a long random string; verifying the build logs show pip install succeeding; verifying the release phase ran `flask db upgrade` successfully; verifying the deploy succeeded and a public URL was assigned; hitting that public URL's `/api/v1/health` endpoint from a browser. The document also covers reading live logs in the Railway dashboard, restarting a deploy, and rolling back to a previous deploy.

One important warning to include in the document: Railway's PostgreSQL `DATABASE_URL` sometimes uses the `postgres://` scheme while SQLAlchemy 2.x requires `postgresql://` — if the deploy fails at startup with a dialect error, the fix is to normalize the scheme inside the Config class. Add that normalization to `server/app/config.py` defensively: if `DATABASE_URL` starts with `postgres://`, rewrite it to `postgresql://` before passing to SQLAlchemy.

Backend requirements remain in force:

- Flask app
- CORS enabled
- environment variables (NOT hardcoded DB credentials)
- Railway-compatible deployment
- uses `DATABASE_URL`

Commit migration scaffold and Procfile changes as "backend(deploy): migrations, release phase, Railway docs".

Definition of Done: The human follows `docs/DEPLOY_RAILWAY.md`, ends up with a live Railway service whose `/api/v1/health` returns 200 over HTTPS. The Railway Postgres instance contains all tables (verifiable via Railway's database query console or by connecting `psql` to the public connection string and running `\dt`). Locally, the alembic migration applies cleanly and the dev SQLite database matches.

Part 8 — Auth: Register, Login, JWT, Role-Based Decorators

Stage: Backend **Goal:** Implement user registration (with role-specific fields), login, current-user endpoint, password hashing, JWT issuance and validation, and role-based access decorators that protect every subsequent route. **Prerequisites:** Parts 5-7.

Verification of Part 7.

- ☐ Public Railway URL (note it down — call it `RAILWAY_URL`) responds to `/api/v1/health` with 200.
- ☐ Connecting to Railway Postgres via the dashboard's data tab and running `\dt`-equivalent shows all eleven tables.
- ☐ All env vars are set on the Railway service: `DATABASE_URL` (auto), `SECRET_KEY`, `JWT_SECRET_KEY`, `FLASK_ENV=production`, `CORS_ALLOWED_ORIGINS`, `PORT` (Railway-managed).
- ☐ Local alembic migration applies and produces equivalent tables on `server/dev.db`.
- ☐ `docs/DEPLOY_RAILWAY.md` exists and is accurate against what the human actually clicked through.
- ☐ The postgres-vs-postgresql scheme normalization is in `server/app/config.py`.

Recommended model: Claude Opus 4.7 (alternative: Claude Sonnet 4.5).

Prompt for Windsurf:

Read `docs/API.md` Auth section. Implement the auth subsystem.

Initialize flask-jwt-extended in the app factory with the `JWT_SECRET_KEY` from config, twelve-hour access-token expiry, identity claim equal to the user's primary key (as a string), and an additional claims callback that includes the role. Add error handlers for expired, invalid, and missing tokens that return the standard error envelope from `docs/API.md`.

Create `server/app/routes/auth.py` as a blueprint registered under `/api/v1/auth`.

Implement: register, login, current user.

Register accepts full name, email, password, role, plus role-specific fields. For role student: roll number, department id, semester. For role teacher: employee code, designation. Validate everything (non-empty, email shape, password minimum length 8, role is one of the two allowed values, role-specific fields present and non-empty, department id exists). On success: hash the password with werkzeug, create the User row, then create the Student or Teacher row with the same primary key (joined-table inheritance), all in one transaction.

Return 201 with the user profile (no password hash) and a freshly issued JWT. On validation failure return 422 with field-level details using the documented error envelope.

Login accepts email and password. Look up by email case-insensitively. On success return 200 with the user profile and a JWT. On unknown email or wrong password return 401 with a generic "invalid credentials" message — never leak which of the two failed.

Current user is GET on `/api/v1/auth/me`, requires JWT, returns the full user profile joined with the role-specific row (student details or teacher details). 401 if no/invalid token.

Create `server/app/utils/auth_decorators.py` with two decorators: `student_required` and `teacher_required`. Each wraps a Flask view, requires a valid JWT, reads the role claim, and returns 403 with the standard error envelope if the role does not match. Add a helper `current_user()` that loads the User by JWT identity and returns the SQLAlchemy instance, raising a clean 401 if not found.

Create a temporary diagnostic route at `/api/v1/_diag/teacher-only` and `/api/v1/_diag/student-only` so the human can verify decorators in Part 8 verification.

These will be removed in Part 12 — leave a TODO comment naming the prompt that removes them.

Backend requirements remain in force:

- Flask app
- CORS enabled
- environment variables (NOT hardcoded DB credentials)
- Railway-compatible deployment
- uses DATABASE_URL

Do not yet implement any teacher- or student-resource routes — those are Parts 9–11.

Commit as "backend(auth): register, login, JWT, role decorators".

Definition of Done: Against the local server: registering a teacher then logging in returns a JWT. Hitting `/api/v1/auth/me` with that token returns the teacher profile. Hitting `/api/v1/_diag/teacher-only` with that token returns 200; with a student token returns 403; with no token returns 401. Same against the Railway URL (push, redeploy, verify).

Part 9 — Course & Enrollment Endpoints

Stage: Backend **Goal:** Implement teacher-side course CRUD and enrollment management, plus student-side course listing. **Prerequisites:** Part 8.

Verification of Part 8.

- ☐ POST `/api/v1/auth/register` with full teacher payload returns 201 with token.
- ☐ POST `/api/v1/auth/register` with a duplicate email returns 422 with a field-level error on email.
- ☐ POST `/api/v1/auth/register` with password length 5 returns 422.

- ☐ POST `/api/v1/auth/login` with correct credentials returns 200 with token; with wrong password returns 401 with generic message.
- ☐ GET `/api/v1/auth/me` with valid teacher token returns the joined teacher profile (with `employee_code` and `designation`).
- ☐ GET `/api/v1/_diag/teacher-only` with student token returns 403; with no token returns 401.
- ☐ All of the above pass against `RAILWAY_URL` after pushing this part.

Recommended model: Claude Sonnet 4.5 (alternative: Claude Opus 4.7).

Prompt for Windsurf:

Read `docs/API.md` Courses section. Implement two blueprints under `/api/v1`.
Departments and helpers (`server/app/routes/departments.py`): GET list of departments (public, no auth required — students need this for registration). GET department by id. GET list of students (`teacher_required`) with optional query filters by department id and semester. GET list of teachers (`teacher_required`). All list endpoints support pagination per the documented convention.
Courses and enrollments (`server/app/routes/courses.py`): `teacher_required` for create / update / delete / enroll / unenroll endpoints; both roles allowed for list-my-courses and get-course-detail (with role-aware authorization inside the handler).
POST course: title, course code. The owning teacher is implicit from the JWT (do not accept `teacher_id` in the body — that prevents cross-tenant abuse). Validate course code uniqueness; return 422 on conflict. Return 201 with the created course.
PATCH course id: only the owning teacher can edit. 403 otherwise. Allow updating title and course code (with uniqueness re-check).
DELETE course id: only the owning teacher. 409 if any sessions exist for any of the course's exams (data integrity — do not orphan sessions).
GET courses (list-my-courses): for teacher returns courses where teacher is the owner; for student returns courses the student is enrolled in via Enrollment. Each list item includes id, code, title, owner teacher's name (for student view) or enrolled student count (for teacher view).
GET course id (detail): teacher must own it OR student must be enrolled in it; otherwise 403. Includes course fields plus a brief list of its exams (id, title, `is_active`) and — for the teacher view only — the enrolled students.
POST course id enroll: `teacher_required`, must own course. Body contains a student email. Look up the student; if not found return 404; if already enrolled return 409. Otherwise create the Enrollment row. Return 201 with the enrollment.
DELETE course id enroll student id: `teacher_required`, must own course. Removes the enrollment. 404 if not enrolled.
GET course id students: `teacher_required`, must own course. Returns enrolled students. Implement all list endpoints with pagination (page and page_size query params). Use marshmallow schemas for serialization (in `server/app/services/schemas/`). Reuse the auth decorators from Part 8. Use the standard error envelope everywhere.
Backend requirements remain in force:

- Flask app
- CORS enabled
- environment variables (NOT hardcoded DB credentials)
- Railway-compatible deployment
- uses DATABASE_URL

Commit as "backend(courses): teacher CRUD, enrollments, role-aware listings".

Definition of Done: A teacher can create a course, enroll a student by email, list their courses with student counts, and remove an enrollment. A student sees only courses they're enrolled in. Cross-tenant attempts (teacher A editing teacher B's course) return 403. All endpoints work locally and on Railway.

Part 10 — Exam, Question, Session, Answer Endpoints

Stage: Backend **Goal:** Implement teacher-side exam/question CRUD with nested questions, exam activation, student-side active-exam discovery, session start with server-authoritative deadline, save-answer (upsert), submit (auto-grade MCQs), and time-remaining endpoints.

Prerequisites: Part 9.

Verification of Part 9.

- ☐ As a teacher, POST course, then POST course id enroll with the seeded student email — returns 201.
- ☐ GET courses as teacher returns the new course with enrolled student count 1.
- ☐ As that student, GET courses returns the course.
- ☐ GET course id as a different student returns 403.
- ☐ DELETE course with no exams returns 204; DELETE course referenced by an existing session would return 409 (no sessions yet, so this is a contract check only — re-test in Part 13).
- ☐ All endpoints work against `RAILWAY_URL` after deploy.

Recommended model: Claude Opus 4.7 (alternative: Claude Sonnet 4.5).

Prompt for Windsurf:

Read `docs/API.md` Exams and Sessions sections. Implement two blueprints under `/api/v1`: exams (`server/app/routes/exams.py`) and sessions (`server/app/routes/sessions.py`). Implement service-layer logic in `server/app/services/exam_service.py` and `server/app/services/session_service.py` so route handlers stay thin.

Exams (teacher_required for create/update/delete/activate/deactivate; student-readable only when active and student is enrolled in the course).

POST exam: course id, title, duration in minutes, total_marks (computed from questions sum but accepted explicitly for sanity check), and a nested list of questions. Each question carries prompt, type (mcq or short_answer), marks, and (for mcq) the four options and the correct

option letter. Validate: course is owned by current teacher; duration > 0; questions list non-empty; for each mcq, all four options present and correct option is one of A/B/C/D; for short_answer, options and correct option must be absent. The created exam starts as is_active=false. Persist exam and questions in one transaction. Return 201 with the full exam including questions but with correct_option fields stripped (it doesn't matter for the teacher response, but adopt the always-strip-from-API rule for student endpoints — for teacher endpoints include correct_option).

PATCH exam id: only the owning teacher. Allow editing title, duration, is_active, and replacing the question list (replace, not merge — simpler semantics). Disallow editing if any session for this exam exists in any state other than aborted_vm or aborted_stealth_vm; return 409 with a clear message.

DELETE exam id: only owning teacher. 409 if any session exists.

POST exam id activate / POST exam id deactivate: only owning teacher. Activation flips is_active true; deactivation false. Activation requires at least one question and a positive duration.

GET exam id: teacher who owns it gets the full exam including correct_option for every mcq. Enrolled student gets the exam ONLY if is_active is true, AND with correct_option fields removed from every question. Other users get 403.

GET course id exams: teacher gets all; enrolled student gets only is_active=true.

GET active exams (`/api/v1/student/active-exams`): student_required. Returns all is_active exams across courses the student is enrolled in. For each, indicate whether the student already has an in_progress, submitted, or aborted session (so the dashboard can render appropriate state).

Sessions (student_required throughout, plus a teacher_required listing route).

POST exam id session (start session): create a new ExamSession with status pre_check, started_at = now, deadline_at = now + duration. If the student already has an in_progress session for this exam, return that one instead of creating a new one (resume semantics). If the student has a submitted session, return 409 — no retakes. If aborted_vm or aborted_stealth_vm exists, allow a new attempt (per the requirement that re-running re-runs gates). The response contains session id, deadline_at (server-authoritative), and the question list with correct_option fields stripped.

POST session id transition: a tiny endpoint that moves a session from pre_check to in_progress once the client has cleared both VM gates. Validates current status is pre_check and that the caller is the session's student. (We'll wire VM-detection clients to this in Parts 20–22.)

PUT session id answer: upsert keyed on (session, question). Body is question id and answer text. Forbid if session status is not in_progress. For mcq questions the answer text must be one of A/B/C/D — validate. Return 200 with no body or a small ack.

POST session id submit: only if session is in_progress. Compute marks for every mcq answer (correct or not, scoring marks for correct, zero for incorrect); short_answer answers get marks_awarded = null pending teacher review. Sum the auto-graded score, persist, set session status submitted, ended_at = now, score = sum (note: teacher manual grading later may increase this score). Return the score breakdown.

GET session id time-remaining: returns seconds until deadline_at, or zero if past. If past and status still in_progress, also flip status to expired and persist before responding (auto-expiry).
GET session id result: student sees own session post-submission with answers, marks_awarded, total. Without correct answers leaked.

Backend requirements remain in force:

- Flask app
- CORS enabled
- environment variables (NOT hardcoded DB credentials)
- Railway-compatible deployment
- uses DATABASE_URL

Commit as "backend(exams,sessions): exam CRUD, activation, session lifecycle, auto-grading".

Definition of Done: Full exam-taking flow works end-to-end via curl: teacher creates exam with questions → teacher activates → student lists active exams → student starts session (pre_check) → client transitions to in_progress (simulating clearing VM gates) → student saves answers → student submits → student fetches result with score. Auto-grading is correct for mcq. Deadline auto-expiry works (test by setting duration = 1 minute and waiting).

Part 11 — Incident Logs, Teacher Reporting, Dummy Data Seed

Stage: Backend **Goal:** Implement the incident-log ingestion endpoint (the live target of all lockdown/VM events), the teacher reporting endpoints (sessions for an exam, full session detail with answers and incidents, manual grading, analytics), and a `/api/v1/_seed` admin endpoint that drops and recreates all data with a configurable shape suitable for demos. **Prerequisites:** Part 10.

Verification of Part 10.

- ☐ Teacher creates an exam with 2 mcq + 1 short_answer questions; response contains the exam with question ids.
- ☐ Teacher activates the exam.
- ☐ Student starts a session, transitions pre_check→in_progress, saves answers (one correct mcq, one incorrect mcq, one short_answer with text), submits. Score equals the marks of the correct mcq only.
- ☐ PUT answer with answer "Z" on an mcq returns 422.
- ☐ PUT answer after submit returns 409.
- ☐ GET session time-remaining when past deadline auto-flips status to expired.
- ☐ All endpoints work against `RAILWAY_URL`.

Recommended model: Claude Opus 4.7 (alternative: Claude Sonnet 4.5).

Prompt for Windsurf:

Three areas in this prompt: incident ingestion, teacher reporting, dummy data seed.

Incident ingestion (`(server/app/routes/incidents.py)`, registered under `(/api/v1)`). POST session id incident: student_required, must own the session, session must be in any non-final state (pre_check, in_progress, expired). Body contains type (must be one of the controlled vocabulary in `(server/app/utils/incident_types.py)`), severity (info, warning, critical), description, and three optional forensic fields cpu_thermal_value, timing_latency_ms, evidence_path. Validate type against the vocabulary; return 422 if unknown. Persist with occurred_at = now (server-side, ignoring any client-provided timestamp to prevent backdating). Return 201 with id and persisted occurred_at.

POST session id incidents (bulk): same auth, accepts a list of incidents in one call, persists all in a single transaction. This is the endpoint the lockdown's offline-queue flusher will use when reconnecting. Return 201 with the list of created ids.

Teacher reporting (`(server/app/routes/teacher_reports.py)`, registered under `(/api/v1/teacher)`, all teacher_required).

GET exams exam id sessions: returns paginated list of sessions for an exam owned by the current teacher. Each item includes session id, student summary (id, name, roll number), status, started_at, ended_at, score, total_marks, and an incident-count plus highest-severity-seen.

GET sessions session id detail: full session view — exam metadata, student profile, every question with the student's answer and marks_awarded (and correct_option for the teacher's eyes), full incident log ordered by occurred_at, and aggregate counts grouped by incident type.

POST sessions session id grade: body contains a list of (question_id, marks_awarded) pairs. For each, the question must be a short_answer in the session's exam, the marks_awarded must be between 0 and the question's max marks. Update Answer.marks_awarded, recompute session score, persist. Return updated score.

GET exams exam id analytics: aggregate stats — sessions count by status, mean/median/min/max scores among submitted, total incidents, top incident types by frequency, percent of submitted sessions that had at least one critical incident. All scoped to the teacher's own exam.

Dummy data seed (`(server/app/routes/admin_seed.py)`, registered under `(/api/v1/_seed)`, gated behind a header check using a `(SEED_TOKEN)` env var so it cannot be hit accidentally on Railway — if the header doesn't match, return 404 to avoid even confirming the endpoint exists).

POST `(/api/v1/_seed)`: drop all rows in dependency order, then generate dummy data using a generator in `(server/seed/generator.py)`. Shape: 2 departments (Computer Science and Information Technology, both at "PUCIT, Lahore"), 3 teachers with realistic Pakistani names and unique employee codes, 6 courses distributed across the 3 teachers (2 per teacher), 30 students with PUCIT-style roll numbers like BSIT-F22-001 through BSIT-F22-030, each student enrolled in 2–4 random courses (not all the same), 2 exams per course with 10 questions each (each exam mixes 7 mcq and 3 short_answer questions), exams alternating is_active true/false so demos always have active and inactive examples. All passwords are the literal string `(pass123)` (document this in the response body so the human can log in immediately). The generator MUST use Faker or equivalent for names but must produce

stable, demoable Pakistani-context content for departments, course titles, and exam titles (e.g., Database Systems, Software Engineering, Operating Systems). Return summary counts plus a credentials block listing one teacher email and one student email with the shared password.

Backend requirements remain in force:

- Flask app
- CORS enabled
- environment variables (NOT hardcoded DB credentials)
- Railway-compatible deployment
- uses DATABASE_URL

Add the SEED_TOKEN env var to `server/.env.example` and document it in `docs/DEPLOY_RAILWAY.md` under a new section on running the seed against the production database. Commit as "backend(reporting,seed): incidents, teacher reports, dummy data".

Definition of Done: A POST to the seed endpoint with the correct token populates the database with the documented shape; the response lists usable demo credentials. Logging in as the demo teacher and hitting the reporting endpoints shows their courses, exams, and (after a student takes one) sessions and incidents. Bulk incident posting works. The seed endpoint without the token returns 404. All endpoints work on Railway.

Part 12 — Tkinter App Skeleton: Window, Theme, Navigation, API Client

Stage: Client **Goal:** Build the foundation of the Tkinter desktop client: app entry point, root window with a centered geometry on a dark theme, screen navigation/router system, theme constants module, reusable UI widgets, and an HTTP client wrapper around `requests` with JWT handling and standardized result tuples. **Prerequisites:** Part 11 (backend complete and live).

Verification of Part 11.

- ☐ POST `/api/v1/_seed` with the correct SEED_TOKEN header returns counts (2 departments, 3 teachers, 30 students, 6 courses, 12 exams, 120 questions) and a credentials block.
- ☐ Without the header, the seed endpoint returns 404 (not 401, not 403).
- ☐ Logging in as the demo teacher and hitting GET teacher exams id analytics for one of their exams returns the analytics envelope with zeros for all aggregates (no sessions yet).
- ☐ Posting a bulk incident with three items against an active session returns 201 with three ids.
- ☐ Manual grading on a short_answer with marks_awarded above the question's max returns 422.
- ☐ All work on `RAILWAY_URL`.
- ☐ The two `_diag` routes from Part 8 are now removed (search for the TODO comment and clean up).

Recommended model: Claude Opus 4.7 (alternative: Claude Sonnet 4.5). Use Opus — getting the navigation and API-client primitives right pays back through every later client prompt.

Prompt for Windsurf:

Build the client foundation under `client/app/`. Use plain Tkinter from the standard library — do not introduce CustomTkinter, Tkinter-Designer, or any other GUI dependency, because PyInstaller compatibility is non-negotiable for the demo. Style everything manually via a theme module.

Theme module at `client/app/ui/theme.py`: define color constants for primary background, secondary background (cards), input background, accent (primary action), accent hover, primary text, secondary text, border, success, error, warning. Define font tuples for heading, subheading, body, small, button. Define spacing constants for large, medium, small padding. Use a deep dark palette (deep navy background, off-white text, a vivid accent for primary actions) — pick exact hex values and stick to them throughout. Document the palette in `docs/ARCHITECTURE.md` under a new "UI theme" subsection.

Reusable widgets module at `client/app/ui/widgets.py`: factory functions returning themed Tk widgets — primary button, secondary button, themed label, themed entry with placeholder support (Tkinter doesn't have native placeholders; implement with focus-in/focus-out bindings), themed combobox, themed text area. Also a `Toast` helper that briefly shows a colored message at the bottom of a window for success/error feedback.

Configuration loader at `client/app/config.py`: reads from environment via python-dotenv (load `client/.env` if present), with these keys — `API_BASE_URL` (default to `http://127.0.0.1:5000/api/v1` for local dev; the human flips it to the Railway URL after Part 18), `SKIP_VM_CHECK` (boolean, default false), `REQUEST_TIMEOUT_SECONDS` (default 10).

API client at `client/app/services/api_client.py`. A class `ApiClient` holding base URL and an in-memory token. Methods `get` / `post` / `put` / `patch` / `delete` each take an endpoint path (relative to base URL) and an optional body, return a tuple of (success: bool, payload: dict-or-None, error: `ErrorInfo`-or-None). `ErrorInfo` carries `http_status` (or None for transport errors), error code from the server envelope, human message, and field-level details if any. The client normalizes failures so the screens never need to catch exceptions: `ConnectionError` becomes (False, None, `ErrorInfo` with code `TRANSPORT` and message "Cannot reach server"), `Timeout` becomes (False, None, `ErrorInfo` with code `TIMEOUT`), non-2xx responses parse the server error envelope. The client automatically attaches the Bearer token if set. Methods `set_token(token)` and `clear_token()` update state. The class is thread-safe enough for the simple usage in this app (no concurrent mutation).

Session/state holder at `client/app/services/session_state.py`: an in-memory store for the currently logged-in user profile and JWT, plus a place for screens to stash transient state across navigation (e.g., the currently selected course). Provide observer-style callbacks so screens can react to logout.

Navigation/router at `client/app/services/router.py`: a `Router` class owning the root Tk window and a content frame that screens are mounted into. Method `show(screen_name, **kwargs)` destroys the current screen, instantiates the named screen class, and packs it. The router holds a registry mapping screen names to classes; screens register themselves in the

router's `__init__` (or via an explicit registration call from `client/app/screens/__init__.py`). The router knows about back-stack semantics for screens that support it.

App entry at `client/app/main.py`. Creates the root Tk window, sets title to "ExamSentinel", sets a fixed size of 1100x720, centers on screen, applies dark background, instantiates the router, instantiates the api client, instantiates session state, registers all screens, navigates to the splash screen (Part 13 will create it; for now create a placeholder splash screen module that just shows a centered "ExamSentinel" label so this part runs).

Provide a small `client/run_dev.bat` that creates a venv if missing, installs `client/requirements.txt`, copies `.env.example` to `.env` if missing, and runs `python -m client.app.main`.

Cross-version Windows compatibility: the foundation here uses only stdlib Tkinter and `requests` — no Windows-specific calls in this part. Avoid any OS-specific assumptions.

Commit as "client(skeleton): app entry, theme, widgets, router, api client".

Definition of Done: Running `client/run_dev.bat` opens an 1100x720 dark-themed window centered on the screen titled "ExamSentinel" showing the placeholder splash. Closing the window terminates the process cleanly. The api client can be invoked from a Python REPL against the local server's `/api/v1/health` and returns (True, payload, None).

Part 13 — Splash, Login, Register Screens

Stage: Client Goal: Implement the three first screens the user sees: a splash that pings the server, a login screen, and a register screen with role-aware fields. **Prerequisites:** Part 12.

Verification of Part 12.

- ☐ `run_dev.bat` opens the dark themed window with the placeholder splash centered.
- ☐ Window size is 1100x720, centered.
- ☐ From a Python REPL: `from client.app.services.api_client import ApiClient; c = ApiClient("http://127.0.0.1:5000/api/v1"); ok, data, err = c.get("/health")` returns `ok=True` with the documented health body.
- ☐ Stopping the server: same call returns `ok=False` with `err.code == "TRANSPORT"`.
- ☐ No CustomTkinter or other GUI lib is present in `client/requirements.txt`.

Recommended model: Claude Sonnet 4.5 (alternative: Claude Opus 4.7).

Prompt for Windsurf:

Implement three screens under `client/app/screens/`.
Splash screen (`splash.py`). On show, displays a large centered "ExamSentinel" wordmark with the accent color, a subtitle "Locked Exam Environment", and a status line. Below the status line a small spinner (animate a rotating Unicode glyph or a sequence of dots — keep it stdlib-only). On mount it kicks off a background thread that hits `/health`. On success: status flips to "Server reachable" briefly, then router navigates to the login screen. On failure: status

flips to a red error message and a "Retry" button appears that re-runs the check. All UI updates from the background thread go through `root.after(0, ...)` to stay on the main thread.

Login screen (`login.py`). A centered card 480px wide on the dark background. Card contains: heading "Sign In", email entry (themed, with placeholder), password entry (with show/hide toggle implemented as a small text button next to it), error label that's hidden by default and shown red when login fails, primary "Login" button full width, a subtle link-styled label "New here? Create an account" that navigates to register. On Login click: validate non-empty fields, set the button to a "Signing in..." disabled state, kick off a background POST to `/auth/login`. On success: store token in api client and user profile in session state, navigate to teacher dashboard or student dashboard depending on role. On failure: show the server's human message in the error label, restore the button. Pressing Enter in the password field triggers Login. Keyboard focus starts on email.

Register screen (`register.py`). Card layout, slightly taller. Common fields at top: full name, email, password, confirm password, role selector (radio buttons or a segmented toggle — student / teacher). Below that, a dynamic role-specific section that swaps based on the selected role: for student it shows roll number, department selector (combobox populated from a GET `/departments` call on screen mount), semester (numeric entry); for teacher it shows employee code and designation. Validation rules inline: passwords match, password length ≥ 8 , valid email shape, role-specific required fields present. Submit button kicks off a background POST to `/auth/register` with the appropriate payload. On success: stash the returned token and profile, navigate directly to the appropriate dashboard (no need to log in again). On failure: render server-returned field-level details next to the corresponding fields. Both login and register screens display a small footer line: "PUCIT FYP 2026". Both screens use only the themed widgets from Part 12.

Cross-version Windows compatibility: pure Tkinter; no OS-specific calls. Threading uses `stdlib threading` and `root.after`.

Commit as "client(auth screens): splash, login, register".

Definition of Done: Launching the client lands on splash → after a successful health ping, navigates to login → typing the seed teacher credentials logs in and (the next prompt's screen will catch the navigation; for now navigate to a placeholder teacher screen that just shows "Logged in as <name>"). Wrong password renders the server's error inline. The register screen swaps fields when role is toggled. Stopping the server during login produces a clean transport error message.

Part 14 — Student Dashboard

Stage: Client **Goal:** Implement the student dashboard: enrolled courses listing, available active exams listing with state badges, exam history. **Prerequisites:** Part 13.

Verification of Part 13.

☐ Splash → server up → navigates to login.

- ☐ Splash → server down → shows red error and Retry; clicking Retry after starting the server proceeds.
- ☐ Login with seed teacher email and `pass123` succeeds and lands on the placeholder teacher screen.
- ☐ Login with wrong password shows the server's "invalid credentials" message inline.
- ☐ Register form swaps role-specific fields when role toggles.
- ☐ Register with mismatched passwords shows inline validation error before any network call.
- ☐ Successful registration auto-logs-in.

Recommended model: Claude Sonnet 4.5 (alternative: Claude Opus 4.7).

Prompt for Windsurf:

Implement the student dashboard at `client/app/screens/student_dashboard.py`. Layout: a left sidebar 240px wide with the user's name and role at top, navigation items below (Courses, Available Exams, History), and a Logout button at the bottom. The remaining area is a content frame that swaps between three sub-views.

Courses sub-view: a vertically scrollable list of cards, one per enrolled course. Each card shows course code, title, and the owning teacher's name. Clicking a card navigates to a course-detail mini-view (in-place inside the content area, with a back arrow) that lists the exams in that course (with each exam's status: active/inactive, and if the student has a session, its status — not_started, in_progress, submitted, expired, aborted_vm, aborted_stealth_vm). On screen mount, fetch GET `/courses` and render. Empty state: a friendly "You are not enrolled in any courses yet" with the secondary text color.

Available Exams sub-view: a list of all currently active exams across the student's courses, fetched from GET `/student/active-exams`. Each row shows exam title, course code/title, duration in minutes, and a state badge derived from the per-student session status the API returns. The primary action button per row is "Start Exam" if the student has no active or final session, "Resume" if there's an in_progress session, "View Result" if submitted, "Retry Exam" if aborted_vm or aborted_stealth_vm. Clicking any of these will eventually navigate to the exam-environment integrity verification screen which Part 17 introduces — for now just stub it with `router.show("exam_integrity_check", session_id=...)` and create a placeholder screen that shows the session id.

History sub-view: a paginated list of past sessions across all the student's exams (final states only — submitted, expired, aborted_*). Each row shows date, exam title, status, and score (or em-dash if no score). Clicking a submitted row drills into a result view showing the answers and marks per question (call GET `/sessions/<id>/result`).

The Logout button clears token from api client, clears session state, and routes back to login. All sub-views fetch lazily on first mount; cache results in session state so switching between sub-views doesn't re-fetch unnecessarily; provide a small "Refresh" button on each sub-view header to force reload.

All network calls run in background threads with marshalling back to main via `root.after`.

Cross-version Windows compatibility: pure Tkinter.

Commit as "client(student): dashboard with courses, available exams, history".

Definition of Done: Logging in as the seed student displays their courses, an Available Exams list with at least the active seeded exams, and an empty History (until they submit). Clicking Start Exam navigates to the placeholder exam-integrity-check screen with a session id passed through.

Part 15 — Teacher Dashboard

Stage: Client **Goal:** Implement the teacher dashboard: courses list with management, students per course, exam list per course with create/activate/deactivate controls — but without the deep exam-creation form (Part 16) or review flow (Part 18). **Prerequisites:** Part 14.

Verification of Part 14.

- ☐ Logging in as a seed student lands on the student dashboard.
- ☐ Courses tab shows the student's enrolled courses; each card is clickable and shows that course's exams.
- ☐ Available Exams tab shows active exams with correct state badges.
- ☐ Start Exam button passes a real session id (visible in the placeholder integrity screen).
- ☐ History tab shows the empty-state when no past sessions exist.
- ☐ Logout button returns to login screen and clears the token (verify by trying to navigate back via dev tools — token should be gone).

Recommended model: Claude Sonnet 4.5 (alternative: Claude Opus 4.7).

Prompt for Windsurf:

Implement the teacher dashboard at `client/app/screens/teacher_dashboard.py` mirroring the student dashboard's sidebar layout but with these sections: My Courses, Reports, and Logout.

My Courses sub-view: a header with a "+ New Course" button on the right and a search input on the left filtering the list as you type. The list shows cards for each course owned by the teacher: course code, title, enrolled student count, exam count. Clicking a card opens a course-detail mini-view in-place with a back arrow.

Course detail mini-view has two horizontal tabs: Students and Exams.

Students tab: paginated list of enrolled students with name, roll number, department, semester. A row action "Remove" calls DELETE enrollment and refreshes. Above the list, an "Enroll Student" input takes a student email and an Enroll button. Common errors (already enrolled, student not found) render as inline messages.

Exams tab: list of exams with title, question count, total marks, duration, an active/inactive toggle (calls activate/deactivate endpoints), and two row actions — "Edit" and "Review Sessions". Above the list, a "+ Create Exam" button navigates to the exam creation screen (Part 16). Edit navigates to the same exam creation screen pre-populated with the existing exam (the form supports both create and edit). Review Sessions navigates to the teacher review screen (Part 18 placeholder for now).

" + New Course" button opens an inline modal asking for course code and title, calls POST course, and on success appends to the list.

Reports sub-view: for now, a simple placeholder showing "Detailed reporting coming in Part 18" — we'll fill it in then.

Cross-version Windows compatibility: pure Tkinter.

Commit as "client(teacher): dashboard with courses and student management".

Definition of Done: Logging in as the seed teacher lands on the teacher dashboard. Their courses are listed with correct student counts. Clicking a course opens detail with student list and exam list. Enroll/remove students works against the backend. Toggling an exam's active state works and the backend reflects the change. The Create Exam button navigates to a placeholder Part 16 screen.

Part 16 — Exam Creation / Edit Flow

Stage: Client **Goal:** Implement a single exam creation/edit screen with a dynamic question builder supporting MCQ and short-answer types, validation, and POST/PATCH to the server.

Prerequisites: Part 15.

Verification of Part 15.

- ☐ Teacher dashboard shows their courses with accurate counts.
- ☐ Course detail Students tab supports enrollment and removal end-to-end.
- ☐ Course detail Exams tab shows existing exams with active toggles that reach the backend.
- ☐ "+ New Course" creates a course; new card appears.
- ☐ Search filters courses live.

Recommended model: Claude Opus 4.7 (alternative: Claude Sonnet 4.5). Opus pays back here — the dynamic form is the most fiddly UI in the app.

Prompt for Windsurf:

Implement `client/app/screens/exam_editor.py`. The screen is reused for both creating a new exam and editing an existing one. Accept route parameters: `course_id` and an optional `exam_id`. If `exam_id` is provided, load the exam on mount and populate fields; otherwise start blank.

Top section: exam title entry, duration in minutes (numeric entry, validated > 0), and a read-only display of the parent course (course code and title). A small text helper notes that activation is done from the course detail Exams tab, not here.

Middle section: a vertically scrollable area listing question editors. Above the list, two buttons: "+ Add MCQ" and "+ Add Short Answer". Each question editor is a card with: an order indicator (auto-numbered), a remove button (small, top-right), a prompt text area, a marks integer entry. For MCQ specifically, four option text entries labeled A/B/C/D and a "Correct Option" combobox with values A/B/C/D. For Short Answer, only the prompt and marks fields. Order can be changed with up/down arrows on each card. The list scrolls; the buttons stay visible at the top of the section.

Bottom section: a Save button (right), a Cancel button (left, returns to course detail). Below the save button a small "Total marks: NN" computed live from the sum of question marks. Validation: title non-empty, duration > 0, at least one question, every question has prompt and marks > 0, every MCQ has all four options non-empty and a correct option selected. Inline validation messages on each invalid field. The Save button is disabled while validation fails; hovering it shows a tooltip explaining what's missing.

On Save: serialize the exam payload (questions in current order via `order_index`), call POST `/exams` for create or PATCH `/exams/<id>` for edit. On success, navigate back to the course detail Exams tab and toast "Exam saved". On failure, render server-side validation errors next to fields and show a toast for top-level errors.

Loading existing exam: if `exam_id` was provided, fetch GET `/exams/<id>` (teacher view returns full questions including `correct_option`), populate the form. If the exam already has any session in any state other than `aborted_*`, the backend will reject edits — render this as a banner at the top in warning color: "This exam has been attempted and cannot be modified. To change it, create a new exam." with the form fields disabled.

Cross-version Windows compatibility: pure Tkinter.

Commit as "client(teacher): exam create/edit screen with dynamic question builder".

Definition of Done: Teacher can create a new exam with mixed MCQ and short-answer questions, save, and see it in the course detail. Editing an existing exam loads existing questions correctly. Adding/removing/reordering questions updates the form smoothly. Trying to edit an exam that already has a session shows the banner and disables fields.

Part 17 — Exam-Taking Screen (UX Only, No Lockdown Yet)

Stage: Client **Goal:** Implement the student exam-taking screen with question navigation, answer auto-save, server-authoritative countdown, submit confirmation, and post-submit result view. The integrity-check screen mounted before this is a stub for now — VM detection and lockdown wire in during Phase J/K/I respectively. **Prerequisites:** Part 16.

Verification of Part 16.

- ☐ Creating a new exam with 7 MCQ + 3 short-answer questions saves and appears in the course detail Exams tab.
- ☐ Editing the exam reorders a question and saves; the new order persists after refresh.
- ☐ Adding an empty MCQ option triggers inline validation.
- ☐ Trying to edit an exam with at least one session in a non-aborted state shows the warning banner and the Save button is disabled.

Recommended model: Claude Opus 4.7 (alternative: Claude Sonnet 4.5). Opus — this is the most demo-critical screen.

Prompt for Windsurf:

Implement two screens.

First, `client/app/screens/exam_integrity_check.py` — a simple placeholder that displays "Integrity checks will run here. Click Continue to proceed." with a Continue button and a Cancel button. Continue calls POST `/sessions/<id>/transition` (the `pre_check` → `in_progress` endpoint from Part 10) and on success navigates to the exam taking screen with the session id. Cancel returns to the student dashboard. We'll replace the body of this screen in Parts 20 and 22 with real VM and stealth-VM checks; the Cancel/Continue contract stays. Second, the meat: `client/app/screens/exam_taking.py`. This screen MUST be structured so a future lockdown subsystem can hook in by calling two functions:

`start_lockdown(self, on_violation)` invoked when the screen mounts in active mode, and `stop_lockdown(self)` invoked when the screen unmounts (any reason — submit, expiry, error, window close). Define these as no-op stubs in this part; Parts 23–27 fill them in. The screen also exposes a `report_violation(type, severity, description, **forensics)` method that posts to the bulk incidents endpoint with offline queuing (next paragraph).

Layout: header bar at top with the exam title, the student's name, and a prominent server-authoritative countdown timer (mm:ss) on the right that polls GET `/sessions/<id>/time-remaining` every five seconds and decrements locally between polls (so the visible tick is one second but the truth is server-authoritative). Below the header, the main area is split: a left navigator panel (180px wide) showing question numbers as buttons that color-code answered (filled accent), unanswered (outline), and current (highlighted) — clicking a number jumps to that question. The right is the question pane: prompt at top, answer area below. For MCQ: four option rows as radio buttons labeled A. / B. / C. / D. with the option text. For short answer: a multi-line text area (8 visible rows). At the bottom, Previous / Next buttons and a primary "Submit Exam" button on the right.

Auto-save: any answer change schedules a debounced PUT `/sessions/<id>/answer` after 1 second of inactivity OR immediately when the user navigates to a different question. A small inline indicator near the timer shows save state ("Saving...", "Saved", or "Save failed — will retry").

Submit: clicking Submit opens a confirmation modal listing answered count, unanswered count, and a final warning that submission is irreversible. Confirm calls POST `/sessions/<id>/submit`. Cancel closes the modal.

Auto-submit on timeout: when timer hits zero, the screen disables inputs, posts submit automatically, and waits for the server to confirm (status will already be flipping server-side via Part 10's auto-expiry, so this is a no-op race that succeeds either way — handle the 409 gracefully if the session is already expired/submitted, treating it as effectively submitted).

Post-submit result: replace the screen contents with a result view — score, total marks, percentage, per-question breakdown showing the student's answer and marks_awarded (where available — short_answers may show "Pending teacher review"). A "Back to Dashboard" button returns to the student dashboard.

Offline incident queue: `report_violation` first attempts an immediate POST to the single-incident endpoint; on transport failure it appends to an in-memory list. A background flusher every 10 seconds attempts to POST any queued items via the bulk endpoint; on success it clears them. On submit, the flusher runs synchronously once before the submit POST so the teacher's view captures everything. The queue is intentionally not persisted to disk — the

lockdown enforces full focus, so a crash that loses the queue is a rare edge case acceptable for the demo.

Cleanup: on every exit path (submit success, submit failure dialog, window close, navigation away), `stop_lockdown` is called and the timer polling thread is signaled to stop. Use a

`threading.Event` for shutdown signaling.

Cross-version Windows compatibility: this screen uses pure Tkinter — Windows-specific lockdown is added later. Avoid OS-specific behavior in this part.

Commit as "client(student): exam taking UX with auto-save and timer (no lockdown yet)".

Definition of Done: Student can take an exam end-to-end without any lockdown: navigate questions, change answers, see auto-save indicator, submit, see results with per-question breakdown. Timer counts down, auto-submits on zero. The integrity check screen's Continue button cleanly transitions the session and lands on the exam taking screen. `start_lockdown` and `stop_lockdown` are wired into mount/unmount as no-op stubs. `report_violation` is callable and posts to the backend successfully.

Part 18 — Teacher Review & Grading Screen

Stage: Client **Goal:** Implement teacher-facing reporting: list of sessions for an exam, deep-dive into one session showing all answers and the full incident timeline, manual short-answer grading interface, and analytics. **Prerequisites:** Part 17.

Verification of Part 17.

- ☐ Student logs in, picks an active exam, integrity-check screen renders, Continue transitions session and opens exam taking screen.
- ☐ Student answers an MCQ; the inline indicator shows "Saved" within ~1 second.
- ☐ Question navigator highlights current question and color-codes answered/unanswered correctly.
- ☐ Submit confirmation modal shows correct counts.
- ☐ On submit, result view shows per-question breakdown with computed score (auto-graded MCQs only; short-answers say "Pending teacher review").
- ☐ Setting duration to 1 minute and waiting causes auto-submit. Server reflects status submitted (or expired-then-submitted).
- ☐ During the exam, calling `report_violation` from a debug breakpoint posts and appears in the database.

Recommended model: Claude Sonnet 4.5 (alternative: Claude Opus 4.7).

Prompt for Windsurf:

Implement two screens, `client/app/screens/teacher_sessions_list.py` and `client/app/screens/teacher_session_detail.py`, plus integrate them into the teacher dashboard.

Sessions list screen takes an `exam_id`. On mount fetches GET

`/teacher/exams/<id>/sessions` paginated. Renders a table-like list: row per session showing student name, roll number, status badge, started_at, ended_at (or em-dash), score / total_marks, incident count, and the highest severity color-coded. Clicking a row opens the session detail screen.

Session detail screen takes a `session_id`. On mount fetches GET

`/teacher/sessions/<id>/detail`. Layout has three regions:

Top: student profile + exam metadata + status + score summary.

Middle (left tab "Answers", right tab "Incidents"). Answers tab lists every question with the student's answer, the correct answer (visible to teachers), and marks_awarded. For short-answer questions, an inline editable marks_awarded field with a "Save Grade" button that calls POST `/teacher/sessions/<id>/grade`. Saving updates the session score live in the top region.

Incidents tab: a chronological list of incidents with timestamp, type, severity (color-coded), description, and the optional forensic columns (cpu_thermal_value, timing_latency_ms, evidence_path) shown when present. A small filter bar at top lets the teacher filter by type or severity.

Add an Analytics drawer accessible from the sessions list screen via a "View Analytics" button — fetches GET `/teacher/exams/<id>/analytics` and shows the aggregate stats (counts by status, score distribution, incident type histogram).

Wire the teacher dashboard's Course Detail Exams tab "Review Sessions" button to navigate to the sessions list screen for that exam, and replace the "Reports" sidebar placeholder with a screen that lists the teacher's exams across all courses with quick links into each one's sessions list.

Cross-version Windows compatibility: pure Tkinter.

Commit as "client(teacher): sessions list, session detail, manual grading, analytics".

Definition of Done: A teacher can pick an exam, see all sessions, drill into one session, view the full Q&A breakdown, manually grade short-answer questions (and watch the score update), and view the incident timeline with all forensic fields. The Analytics drawer renders aggregates. The full demo flow now works end-to-end without security: student takes exam → teacher reviews submission and incidents.

Part 19 — Standard VM Detection Module

Stage: Security **Goal:** Build the standard VM detection module as a pure-detection library with no UI, returning a structured verdict that includes type, confidence, and a list of indicators. No integration into the app yet. **Prerequisites:** Part 18 (full happy path demo works).

Verification of Part 18.

- ☐ Teacher dashboard sidebar Reports section lists all exams across all courses with quick links.
- ☐ Sessions list shows all sessions for a chosen exam, including the demo student's submitted session from Part 17.

- ☐ Session detail Answers tab allows manual grading of short-answer questions; saving updates the score in the top region without a full refresh.
- ☐ Incidents tab shows incidents posted via `report_violation` from Part 17 verification.
- ☐ Analytics drawer renders correctly with non-zero values now that there is at least one session.

Recommended model: Claude Opus 4.7 (alternative: Claude Sonnet 4.5). Opus — the heuristics are subtle and library specifics matter.

Prompt for Windsurf:

Read `docs/SECURITY_DESIGN.md` Standard VM Detection section. Build `client/app/vm_detect/standard.py` as a pure detection module — no UI, no logging side effects beyond what the function returns to the caller.

Public API: a single function `detect_standard_vm()` that returns a structured result containing: `is_vm` boolean, `vm_type` one of "vmware" / "virtualbox" / "hyperv" / "qemu" / null, `confidence` one of "high" / "medium" / "none", `indicators` a list of structured findings each with category, name, evidence (a short string describing the matched value), and weight (1 for soft, 2 for strong). Confidence aggregation rule: total weight ≥ 4 OR any single weight-2 indicator $\rightarrow$ high; total weight 1-3 $\rightarrow$ medium; 0 $\rightarrow$ none. `is_vm` is true for medium or high.

Detector implementations as separate modules under `client/app/vm_detect/standard_detectors/`: one module per detection method, each exposing a function returning a list of indicators (possibly empty).

Process detector: uses `psutil.process_iter` to scan running process names against a known set — `vmtoolsd.exe`, `vmwaretray.exe`, `vmwareuser.exe`, `VBoxService.exe`, `VBoxTray.exe`, `prl_tools.exe`, `qemu-ga.exe` — case-insensitive match on the process executable basename. Each match is a weight-2 indicator with `vm_type` implied by the process.

WMI hardware fingerprint detector: uses the `wmi` library to query `Win32_ComputerSystem` (Manufacturer, Model), `Win32_BaseBoard` (Manufacturer, Product), `Win32_BIOS` (Manufacturer, SMBIOSBIOSVersion), `Win32_VideoController` (Name). Match substrings (case-insensitive) for VMware ("VMware", "VMW"), VirtualBox ("VirtualBox", "VBox", "Oracle Corporation" + "VirtualBox"), Hyper-V ("Microsoft Corporation" + "Virtual Machine", "Hyper-V"), QEMU ("QEMU"). Each match is a weight-2 indicator (BIOS-level matches) or weight-1 (video controller, sometimes innocuous).

MAC address / OUI detector: enumerate active network adapters via WMI `Win32_NetworkAdapter` where `PhysicalAdapter` is true, get `MACAddress`, normalize to the first three octets, and match against known OUI ranges: 00:05:69, 00:0C:29, 00:1C:14, 00:50:56 $\rightarrow$ VMware (weight 2); 08:00:27, 0A:00:27 $\rightarrow$ VirtualBox (weight 2); 00:15:5D $\rightarrow$ Hyper-V (weight 2); 52:54:00 $\rightarrow$ QEMU (weight 2). Skip adapters that have no MAC.

Registry artifact detector: uses `winreg` (stdlib) to probe presence of well-known guest-tools registry keys — `HKLM\SOFTWARE\VMware, Inc.\VMware Tools` (VMware, weight 2); `HKLM\SOFTWARE\Oracle\VirtualBox Guest Additions` (VirtualBox, weight 2); `HKLM\SOFTWARE\Microsoft\Virtual Machine\Guest\Parameters` (Hyper-V, weight 2); `HKLM\HARDWARE\DESCRIPTION\System\BIOS` where `SystemManufacturer` or `SystemProductName` contains the same vendor strings (weight 2). Each present key is one indicator.

Combine all detector outputs, compute confidence, derive `vm_type` as the most-cited type across indicators (ties broken by weight then alphabetical). Return the structured verdict. Resilience: every detector wraps its low-level calls in try/except — a single failing detector must not crash the whole detection. Log to a per-module (`logging.getLogger`) at debug level. Detectors that require admin (none of the above strictly do, but WMI can be flaky in restricted environments) degrade gracefully.

Add a small CLI in (`client/app/vm_detect/__main__.py`): (`python -m client.app.vm_detect standard`) runs the detector and pretty-prints the verdict. This is the dev tool the human uses to test on real hardware vs a VirtualBox guest.

Cross-version Windows compatibility: target Windows 10 and 11 only. Use APIs available on both. Where Windows 10 lacks something, prefer the Windows 10-compatible alternative.

Commit as "security(vm): standard VM detection module".

Definition of Done: Running the CLI on bare metal returns a verdict with confidence "none". Running it inside a default VirtualBox guest returns `is_vm=true`, `vm_type="virtualbox"`, `confidence="high"`, with multiple indicators across processes, WMI, MAC, registry. Same in a default VMware guest with `vm_type="vmware"`. The detector survives running in odd environments (no network adapter, restricted WMI) without crashing.

Part 20 — Standard VM Detection Integration (Pre-Exam Gate)

Stage: Security Goal: Wire (`detect_standard_vm`) into the integrity check screen as the first of two pre-exam gates, with proper user messaging, incident logging, and session abort handling.

Prerequisites: Part 19.

Verification of Part 19.

- ☐ (`python -m client.app.vm_detect standard`) on the host OS returns confidence "none".
- ☐ Same command inside a freshly-installed VirtualBox guest returns `is_vm=true` with at least three indicators across distinct categories.
- ☐ Same inside a VMware guest returns `is_vm=true` with `vm_type=vmware`.
- ☐ Killing all VMware/VirtualBox processes inside a guest still leaves WMI / MAC / registry indicators detectable (verifies multi-detector independence).
- ☐ A failure injected into one detector (e.g., raise inside the WMI module) does not crash the others — verdict still returned.

Recommended model: Claude Sonnet 4.5 (alternative: Claude Opus 4.7).

Prompt for Windsurf:

Replace the body of (`client/app/screens/exam_integrity_check.py`) from Part 17 with the real check. Keep the route contract: it receives a (`session_id`) and either continues into the exam taking screen or aborts back to the dashboard.

On mount, the screen displays a status panel: the heading "Pre-Exam Integrity Check", a list of two checks ("Virtual Machine — Standard", "Virtual Machine — Stealth"), each with a

circular indicator (pending/running/passed/failed), and a status detail line per check. The Continue button is disabled until both checks pass; the Cancel button is always enabled and aborts back to the dashboard, posting an incident of type EXAM_CANCELLED_BY_USER and informing the server. We'll add the second check in Part 22.

First check (this part): runs `detect_standard_vm` on a background thread on mount. If `SKIP_VM_CHECK` config flag is true, mark the check as "Skipped (dev mode)" and let it count as passed. Otherwise:

If verdict `is_vm` is false: mark passed, status "No virtualization detected".

If verdict `is_vm` is true: mark failed with status "<vm_type> detected (confidence: <confidence>)". Below the status, render the full indicator list as a small expandable details section. Post a bulk incident batch to the server with a single VM_DETECTED incident: type=VM_DETECTED, severity=critical, description summarizing vm_type and confidence, evidence_path null. Then call POST `/sessions/<id>` (we don't have an explicit abort endpoint — call POST submit on a pre_check session to convert it; better: extend Part 10's session API to accept a state transition to aborted_vm — instruct Windsurf to add this endpoint now: POST `/sessions/<id>/abort` with body containing reason in {vm, stealth_vm, user}, only allowed from pre_check, transitions to aborted_vm or aborted_stealth_vm or aborted_by_user). Wait — to keep this part contained, just add that `/abort` endpoint here as a small backend addendum and document it in `docs/API.md`.

Then call abort with reason=vm. Disable Continue permanently. Show a prominent message: "This exam cannot be taken inside a virtual machine. Please run on a physical Windows 10/11 device. Closing and reopening the application will not bypass this check."

The screen handles the case where the second check (stealth) is not yet implemented (Part 22) by treating it as auto-passed for now but greyed out with the note "(implementation pending)".

Backend addendum: add POST `/sessions/<id>/abort` to `server/app/routes/sessions.py`. Accepts JSON with reason in {vm, stealth_vm, user}. Allowed only from pre_check status. Transitions to aborted_vm / aborted_stealth_vm / aborted_by_user. Records an automatic incident of the corresponding type if not already present. Returns 200 with the new status.

Cross-version Windows compatibility: ensure the integrity check screen works even when the underlying detection raises (e.g., on a non-Windows dev machine — though we don't ship for non-Windows, it helps the developer iterate). When `wmi` import fails entirely, the detector should return `is_vm=false` with `confidence=none` and a single indicator noting "wmi_unavailable" so the gate doesn't block development on machines without pywin32 fully working.

Commit as "security(integration): standard VM gate, abort endpoint, integrity check UX".

Definition of Done: Starting an exam on bare metal flows through the integrity check (standard VM passes, stealth pending) and proceeds to the exam taking screen via Continue. Starting an exam inside VirtualBox lands on integrity check, the standard VM check fails with detailed indicators, an incident is posted, the session transitions to aborted_vm on the server, Continue stays disabled, the dashboard shows a "Retry" affordance for that exam (per Part 14). Re-running the .exe and re-trying the same exam re-runs the check and re-fails with another incident — confirmed by checking the teacher's session list shows two aborted_vm sessions.

Part 21 — Stealth VM Detection Module

Stage: Security Goal: Build the stealth VM detection module — heuristics that catch hardened/sanitized VMs where standard guest-tool indicators have been removed or spoofed.

Prerequisites: Part 20.

Verification of Part 20.

- ☐ Bare metal: integrity check first row passes, second row shows "(implementation pending)", Continue enabled.
- ☐ VirtualBox guest: integrity check first row fails with detailed indicators; an incident with type VM_DETECTED severity critical is in the session's incident list; session status is aborted_vm; Continue is disabled.
- ☐ Re-running the .exe inside the VM and trying the same exam creates a second aborted_vm session.
- ☐ POST `/sessions/<id>/abort` with reason=user from pre_check transitions to aborted_by_user; with bad reason returns 422.
- ☐ SKIP_VM_CHECK=true bypasses the standard VM gate entirely and marks it skipped.

Recommended model: Claude Opus 4.7 (alternative: Claude Sonnet 4.5). Opus — ctypes/CPUID/RDTSC code is finicky and Opus produces cleaner first drafts.

Prompt for Windsurf:

Read `docs/SECURITY_DESIGN.md` Stealth VM Detection section. Build `client/app/vm_detect/stealth.py` and supporting submodules under `client/app/vm_detect/stealth_detectors/`. Same return shape as Part 19's detector — `is_vm`, `vm_type` (which here may be "unknown_hypervisor" if vendor string isn't recognized), `confidence`, `indicators`. Same confidence aggregation rule. Public function `detect_stealth_vm()`.

Detectors (each in its own module):

CPUID hypervisor-present bit detector: use ctypes to invoke the CPUID instruction at leaf 1 and inspect ECX bit 31. The cleanest portable approach inside Python on Windows is to allocate a small piece of executable memory via VirtualAlloc with PAGE_EXECUTE_READWRITE, write a short instruction stub that performs CPUID and stores the result, cast to a function pointer with ctypes, and call it. Implement this carefully with proper memory cleanup. If the bit is set, emit a weight-2 indicator "cpuid_hv_bit_set" (vm_type unknown — set later by vendor string).

CPUID hypervisor vendor string detector: same VirtualAlloc trick at leaf 0x40000000, read the vendor string in EBX/ECX/EDX (12 ASCII chars). Match against known strings:

"VMwareVMware" → vmware (weight 2); "VBoxVBoxVBox" → virtualbox (weight 2);

"Microsoft Hv" → hyperv (weight 2); "KVMKVMKVM\0\0\0" → kvm (weight 2);

"TCGTCGTCGTCG" → qemu (weight 2); "XenVMMXenVMM" → xen (weight 2); anything else nonzero → unknown_hypervisor (weight 1).

RDTSC timing detector: invoke RDTSC twice around a CPUID instruction (which traps on most hypervisors). Run several hundred iterations, take the median delta. Compare against a calibrated threshold (the median delta on bare metal is typically under 100 cycles for that loop body; under hypervisor it's commonly 1000+). Use a conservative threshold of 500 cycles to avoid false positives. If exceeded, emit weight-1 indicator (timing alone is noisy, so weight 1). Populate `timing_latency_ms` of the resulting indicator with the measured median converted to milliseconds.

Sleep skew detector: spawn a thread that calls Sleep for a known duration via ctypes Sleep, measures wallclock delta versus a tight QueryPerformanceCounter loop, and looks for skew patterns characteristic of hypervisor scheduling (large positive skew). Soft signal — weight 1. False-positive prone on busy hosts; document this in code comments.

Driver enumeration detector: enumerate loaded kernel drivers via WMI

Win32_SystemDriver and look for known hypervisor-tools drivers that survive guest-tool uninstall (vmci, vmmemctl, vboxguest, vboxsf, vboxvideo, hyperv_vsmb, etc.). Each match weight 2.

Deep registry artifact detector: probe registry keys that survive normal cleanup — HKLM\HARDWARE\DESCRIPTION\System SystemBiosVersion / VideoBiosVersion, HKLM\SYSTEM\ControlSet001\Services where service names contain hypervisor strings. Strict substring matches for vendor strings. Weight 1 (these are weaker, and modern stealth VMs do clean some of them).

Thermal sensor anomaly detector: WMI MSAcpi_ThermalZoneTemperature query. Real machines return varying values across multiple polls; many hypervisors return constant "junk" values (or no thermal zone at all). Take three samples 200ms apart. If thermal zone is missing entirely, emit weight-1 indicator. If three samples are exactly equal AND fall within a known stub range (e.g., exactly 273.1 K = 0 C, or 300.0 K), emit weight-1 indicator. Populate `cpu_thermal_value` of the indicator with the observed reading converted to Celsius.

Combine all detector outputs, compute confidence using the standard rule. Resilience: same as Part 19 — every detector wraps in try/except, a failing detector emits no indicators (not a crash). Detectors that require admin (none strictly do) degrade gracefully.

Pure detection — no UI, no logging side effects.

Add to the CLI: `python -m client.app.vm_detect stealth` runs stealth and pretty-prints. `python -m client.app.vm_detect both` runs both and pretty-prints both verdicts side by side.

Cross-version Windows compatibility: ensure CPUID/RDTSC stubs use byte sequences that work on x86_64 Windows 10 and 11. Document in code comments that this module assumes x64 — refuse to run on x86 (rare on Windows 11 anyway) by checking `platform.machine()` early.

Commit as "security(vm): stealth VM detection module with CPUID/RDTSC heuristics".

Definition of Done: CLI on bare metal: confidence "none", possibly one or two soft indicators (e.g., a noisy timing measurement) but never crossing the medium threshold. CLI inside VirtualBox/VMware/Hyper-V guests with all guest tools uninstalled: confidence "high" with multiple indicators including CPUID hv-bit and vendor string. Module survives unusual environments (missing thermal zone, restricted driver enumeration) without crashing.

Part 22 — Stealth VM Detection Integration & Single-Launch Behavior

Stage: Security Goal: Wire stealth detection as the second integrity check, ensuring both gates must pass before transitioning to `in_progress`, and confirming the "every relaunch re-checks" requirement holds. **Prerequisites:** Part 21.

Verification of Part 21.

- ☐ CLI `(both)` on bare metal: standard "none", stealth "none" (or "medium" with noisy timing — investigate if so).
- ☐ CLI `(both)` inside a VirtualBox guest with guest tools removed: standard's confidence drops (process and registry indicators gone) but stealth catches it via CPUID hv-bit and vendor string.
- ☐ CLI on x86 (if testable) refuses to run with a clear message.
- ☐ Killing the wmi import (rename installed package temporarily) — the detector still runs, with WMI-dependent detectors emitting nothing rather than crashing.

Recommended model: Claude Sonnet 4.5 (alternative: Claude Opus 4.7).

Prompt for Windsurf:

Update `(client/app/screens/exam_integrity_check.py)` to run the stealth check after the standard check completes. Both checks run sequentially (not in parallel — sequential keeps the UX clearer and the stealth detector's timing measurements more stable when not competing with WMI calls).

When standard passes, stealth begins automatically. Update its row to "running" and call `(detect_stealth_vm)` on a background thread. On result:

If `(is_vm)` is false: row passes with status "No hypervisor detected". Continue button enables.

If `(is_vm)` is true: row fails with status "Hypervisor detected (<vm_type>, confidence: <confidence>)" and the indicator details expand. Post a bulk incident with one STEALTH_VM_DETECTED incident: severity critical, description summarizing `vm_type` and confidence and the count of indicators per category. The forensic columns `(cpu_thermal_value)` and `(timing_latency_ms)` MUST be populated from the matching indicators if present (find the first indicator carrying each field and copy it onto the incident). Call POST `(/sessions/<id>/abort)` with reason=stealth_vm. Continue stays disabled. Show: "This exam cannot be taken inside any virtualized environment. Please run on a physical Windows 10/11 device."

Single-launch behavior: confirm and document in code comments that the integrity check runs on every navigation to this screen — there is no caching of the verdict across runs. The .exe relaunching naturally re-runs the check because the screen mounts fresh. Add a tiny module-level cache only within a single screen instance to avoid re-running if the user clicks Continue twice rapidly — but invalidate it on any navigation away.

Add an "Audit Log" subtle link at the bottom of the integrity screen that, after a failed gate, lets the student copy the indicator JSON to clipboard for support purposes (to disprove false positives). Log a CLIPBOARD_USED_AUDIT incident on use — but do NOT block this

clipboard write (this is the only place the app intentionally allows clipboard write before lockdown engages).

Update `(docs/SECURITY_DESIGN.md)` with a short subsection "Forensic field population" explaining which incidents must populate `cpu_thermal_value` and `timing_latency_ms`. Also document the `SKIP_VM_CHECK` behavior in the security design — when true, both gates are skipped but a `STARTUP_DEV_MODE` incident is recorded so the teacher can see this attempt was non-production.

Cross-version Windows compatibility: ensure the screen renders identically on Windows 10 and 11 in default DPI; test (in Part 28 smoke test) that high-DPI doesn't break the layout.

Commit as "security(integration): stealth VM gate, forensic field population, single-launch contract".

Definition of Done: Bare metal: both gates pass, Continue enables, exam proceeds. Inside a stealth-prepped VM (guest tools removed, common artifacts cleaned): standard gate may pass; stealth gate fails with `CPUID`-based indicators, an incident with type `STEALTH_VM_DETECTED` is posted carrying `timing_latency_ms` (and `cpu_thermal_value` if relevant), session aborts, dashboard shows Retry. Closing and reopening the .exe and trying again re-runs both gates — verified by seeing two aborted sessions of the relevant type in the teacher's view.

Part 23 — Lockdown Architecture: Manager, Threading, Violation Pipeline

Stage: Security **Goal:** Build the lockdown manager — the central coordinator that owns all lockdown subsystems, exposes the start/stop/violation contract that the exam taking screen already calls, and guarantees clean teardown under all exit paths. **Prerequisites:** Part 22.

Verification of Part 22.

- ☐ Bare metal demo: integrity check passes both gates, exam proceeds, demo continues to function.
- ☐ Stealth-prepped VM: stealth gate fails, incident posted with populated forensic fields, session aborted_stealth_vm, dashboard offers Retry.
- ☐ Two consecutive failed attempts inside the VM produce two aborted sessions visible to the teacher.
- ☐ Audit Log link copies the indicator JSON to clipboard and posts a `CLIPBOARD_USED_AUDIT` incident.

Recommended model: Claude Opus 4.7 (alternative: Claude Sonnet 4.5). Opus — clean shutdown under exceptions is hard; Opus's reasoning helps.

Prompt for Windsurf:

Read `(docs/SECURITY_DESIGN.md)` Live Lockdown sections. Build the lockdown manager skeleton at `(client/app/lockdown/manager.py)`.

Public class `LockdownManager` with constructor accepting the Tk window handle of the exam taking screen and a `report_violation` callable (provided by the screen — same one from Part 17). Methods: `start()`, `stop()`, `is_active` property. The manager owns a list of subsystems registered at construction time (an ordered list of subsystem objects implementing a tiny protocol: `name`, `start()`, `stop()`, `is_started`). For now no subsystems are registered — Parts 24–26 add them.

`start()` invokes each subsystem's `start()` in order. If any one raises, it logs and continues — but records a `STARTUP_PARTIAL_FAILURE` incident and an in-memory list of failed subsystems. The manager always reaches a coherent state where as many subsystems are active as possible. After all start attempts, `start()` posts a `LOCKDOWN_ENGAGED` incident with severity info containing the list of active subsystems.

`stop()` invokes each subsystem's `stop()` in reverse order. Same fault-tolerance rule — failures are logged but do not prevent other subsystems from stopping. After all stop attempts, posts a `LOCKDOWN_DISENGAGED` incident. `stop()` is idempotent: calling it twice is safe and the second call is a no-op.

Critical contract: `stop()` MUST be invoked under all circumstances — submit, expire, user-cancel, window close, even unhandled exception. The exam taking screen wraps its lifecycle in a try/finally that always calls `stop`. Additionally, the manager registers a `sys.excepthook` and a `tkinter` global error handler installed on construction; the original handlers are saved on construction and restored on stop. This double-belt approach catches stray crashes.

Threading model: each subsystem may use its own thread(s). The manager passes each subsystem a shared `threading.Event` named `shutdown_event`. Subsystems must respect this event for clean shutdown — every loop polls it with a short timeout. Subsystem threads MUST be daemon threads so a forced process exit doesn't hang.

Violation pipeline: provide a thin `report(type, severity, description, **forensics)` method on the manager that subsystems call instead of the screen's report directly. The manager forwards to the screen's `report_violation` after attaching standard metadata (subsystem name, occurred_at via local clock for ordering — server timestamps win on persistence per Part 11).

Add a `client/app/lockdown/__init__.py` exporting `LockdownManager`.

Update `client/app/screens/exam_taking.py`'s `start_lockdown` and `stop_lockdown` stubs to instantiate the manager on start (passing the window and a bound `report_violation` method) and call its start/stop. Also wrap the existing `try/finally` cleanup logic to ensure stop is always called.

Cross-version Windows compatibility: this part is pure Python orchestration with no Windows-specific calls — but the subsystems added next are Windows-only. Document in module docstring that the manager is Windows-targeted and refuses to start on non-Windows by checking `platform.system()`.

Commit as "security(lockdown): manager skeleton, threading model, violation pipeline".

Definition of Done: Starting an exam on bare metal posts `LOCKDOWN_ENGAGED`. Submitting (or aborting via window close, force-killing the dev console, or hitting Cancel) posts `LOCKDOWN_DISENGAGED`. The manager handles a deliberately-injected subsystem failure (write a tiny test subsystem whose start raises) by logging, posting

STARTUP_PARTIAL_FAILURE, and continuing the demo. Calling stop twice is a no-op the second time.

Part 24 — Lockdown: Keyboard Hook

Stage: Security Goal: Implement the low-level keyboard hook subsystem that blocks Alt+Tab, Win key both sides, Ctrl+Esc, Alt+Esc, Ctrl+Shift+Esc, Print Screen, Alt+F4 — while permitting normal typing. **Prerequisites:** Part 23.

Verification of Part 23.

- ☐ Starting the exam posts a LOCKDOWN_ENGAGED incident; submitting posts LOCKDOWN_DISENGAGED.
- ☐ An exception thrown inside the exam screen mid-exam triggers the excepthook path; a LOCKDOWN_DISENGAGED incident is still recorded.
- ☐ Closing the window mid-exam: same — DISENGAGED is recorded.
- ☐ Manager's `stop()` called twice: the second call returns immediately without errors and does NOT post a duplicate DISENGAGED incident.

Recommended model: Claude Opus 4.7 (alternative: Claude Sonnet 4.5). Opus — keyboard hook code is famously fragile.

Prompt for Windsurf:

Implement `client/app/lockdown/keyboard.py`. Class `KeyboardLockdown` implementing the subsystem protocol from Part 23.

Mechanism: install a Windows low-level keyboard hook (WH_KEYBOARD_LL) via `SetWindowsHookExW` from `user32` using `ctypes`. The hook callback is a Python function decorated with the appropriate `WINFUNCTYPE` so Windows can invoke it. The hook runs on the thread that installed it — to keep the main UI thread responsive, install the hook on a dedicated thread that owns a Windows message loop (`PeekMessage / GetMessage / DispatchMessage`). The thread idles in the message loop and exits when the `shutdown_event` from the manager is signaled (use `PostThreadMessage` to wake the loop).

Block list (consume the keystroke by returning 1 from the hook callback so it's not propagated):

- Alt + Tab (any direction)
- Alt + F4
- Alt + Esc
- Ctrl + Esc (Start menu)
- Ctrl + Shift + Esc (Task Manager)
- Win key left (LWIN, vk 0x5B), Win key right (RWIN, vk 0x5C), with or without modifiers
- Print Screen (vk 0x2C)

- Ctrl + Win + any letter (Win key combos in general — match by vk being LWIN/RWIN being currently down)

Allow list: every typing key (letters, digits, punctuation, space, backspace, enter, tab when alone, arrows, home/end). Do NOT block plain Tab — students need it for navigation between form fields.

Modifier state tracking: maintain in-process state for whether Alt, Ctrl, Shift, Win are currently held (toggled on key-down, off on key-up events delivered through the hook).

Decide blocking based on combined state.

Each blocked event posts a `KEYBOARD_BLOCKED` incident with severity warning, description naming the blocked combination (e.g., "Alt+Tab", "PrintScreen"). Throttle posting: do not post more than one identical-description incident per 2 seconds (held-down keys would otherwise spam the server).

Resilience: if `SetWindowsHookExW` fails (admin not granted, some endpoint security software blocking it), the subsystem records an `UNAVAILABLE` incident, sets `is_started` false, and does not crash. Document in code comments that this hook requires Administrator privileges to be reliable on locked-down corporate Windows installs (PyInstaller manifest in Part 28 will request elevation).

Clean shutdown: `UnhookWindowsHookEx` in `stop()`, join the message-loop thread with a 2-second timeout, log if it doesn't exit cleanly. Restoring keyboard state is automatic — Windows discards the hook on unhook.

Register `KeyboardLockdown` in the manager's subsystem list (in Part 27 we will register all of them centrally; for now, in Part 24, instantiate it on the manager directly so we can test).

Cross-version Windows compatibility: `WH_KEYBOARD_LL` is identical on Windows 10 and 11. No version-specific API calls. Use only documented user32 functions.

Commit as "security(lockdown): low-level keyboard hook subsystem".

Definition of Done: During an exam, pressing Alt+Tab, Win, Ctrl+Esc, Ctrl+Shift+Esc, Alt+F4, or Print Screen has no effect — the exam window stays focused and full-screen capture-suppression at minimum prevents the keyboard-driven Print Screen capture. Each blocked event produces a `KEYBOARD_BLOCKED` incident on the server (with throttling — holding Alt+Tab does not produce 100 incidents per second). Submitting cleanly removes the hook (verifiable: after exam, Alt+Tab works again, Win key opens Start menu).

Part 25 — Lockdown: Process Killer, Clipboard Scrubber, Right-Click Suppression

Stage: Security **Goal:** Three smaller subsystems: scan/kill blacklisted processes, clear the clipboard on a short interval, and suppress right-click in the exam screen. **Prerequisites:** Part 24.

Verification of Part 24.

- ☐ Exam in progress: Alt+Tab, Win, Ctrl+Esc, Ctrl+Shift+Esc do nothing visible.
- ☐ Each blocked event produces an incident; holding the key produces at most one incident every 2 seconds.
- ☐ Submitting and returning to dashboard restores all keys to normal behavior.

☐ Running without admin produces an UNAVAILABLE incident and the subsystem reports `is_started=false`; the rest of lockdown still works.

Recommended model: Claude Sonnet 4.5 (alternative: Claude Opus 4.7).

Prompt for Windsurf:

Three modules under `client/app/lockdown/`.

`process_kill.py`: class `ProcessKillSubsystem`. Maintains a blacklist of process executable basenames in a constant set: `chrome.exe`, `msedge.exe`, `firefox.exe`, `brave.exe`, `opera.exe`, `vivaldi.exe`, `code.exe`, `cursor.exe`, `windsurf.exe`, `sublime_text.exe`, `notepad++.exe`, `discord.exe`, `telegram.exe`, `whatsapp.exe`, `signal.exe`, `slack.exe`, `zoom.exe`, `teams.exe`, `skype.exe`, `anydesk.exe`, `teamviewer.exe`, `ultraviewer.exe`, `rustdesk.exe`, `parsec.exe`, `snippingtool.exe`, `screenclip.exe`, `snagiteditor.exe`, `snagit32.exe`, `obs64.exe`, `obs32.exe`, `sharex.exe`, `screenrec.exe`, `lightshot.exe`, `cmd.exe`, `powershell.exe`, `pwsh.exe`, `wt.exe`, `taskmgr.exe`, `regedit.exe`, `msconfig.exe` — all case-insensitive. Background thread polls `psutil.process_iter` every 2 seconds while `shutdown_event` is not set. Any blacklisted match: kill via `psutil`'s `terminate()`; if still alive after 200ms, `kill()`. Post a `BLACKLISTED_PROCESS_KILLED` incident with severity critical, description naming the process and pid. Throttle: do not post more than one incident per (process_name, pid) pair (a relaunched same name with different pid is a new incident). Keep the blacklist set as a module constant so Part 30's docs can reference it.

`clipboard_scrub.py`: class `ClipboardScrubSubsystem`. Background thread that, every 500ms while `shutdown_event` is not set, opens the clipboard via `pywin32` `OpenClipboard`, calls `EmptyClipboard`, then `CloseClipboard`. The first time clipboard contained data, post a `CLIPBOARD_SCRUB` incident with severity warning describing what format was present (`CF_TEXT`, `CF_BITMAP`, etc. — read with `EnumClipboardFormats` before clearing).

Suppress further incidents for 10 seconds to avoid spam if the clipboard is being repeatedly populated. Resilience: clipboard access can be temporarily denied by other apps holding the clipboard open — catch and skip silently, retry next tick.

`right_click_suppress.py`: class `RightClickSuppressSubsystem`. Doesn't run a thread. On `start()`, walks the entire Tkinter widget tree from the exam window and binds `<Button-3>` and `<Control-Button-1>` (Mac-style right-click, harmless on Windows but cheap) to a do-nothing-and-return-break handler at the widget level. On `stop()`, unbinds. New widgets created mid-exam (e.g., the submit confirmation dialog) won't auto-inherit; expose a small helper `bind_for_widget(widget)` that the exam screen calls when creating dynamic widgets. Post a `RIGHT_CLICK_BLOCKED` incident on every blocked attempt with throttling (one per 5 seconds).

Register all three in the manager. Each implements the subsystem protocol. Each is independently fault-tolerant per Part 23.

Cross-version Windows compatibility: `psutil` and `pywin32` work identically on Windows 10 and 11. Tkinter binding is OS-agnostic. No version-specific code.

Commit as "security(lockdown): process killer, clipboard scrubber, right-click suppression".

Definition of Done: Launch `chrome.exe` during an exam — it dies within ~2 seconds, an incident appears in the teacher's view. Copy text from the exam answer area (using only the keyboard,

since right-click is blocked) — within 500ms the clipboard is empty; an incident appears once. Right-click in the exam window does nothing. After submit, all three subsystems are stopped: chrome.exe stays alive, clipboard works, right-click works.

Part 26 — Lockdown: Fullscreen, Focus Monitor, Mouse Boundary, Multi-Monitor

Stage: Security **Goal:** Implement the visual/spatial lockdown layer — true fullscreen takeover, taskbar hiding, focus loss detection with re-foregrounding, mouse cursor confinement, and multi-monitor blocking. **Prerequisites:** Part 25.

Verification of Part 25.

- ☐ Process killer kills launched chrome.exe within ~2 seconds during an exam, with an incident posted.
- ☐ Pasting into the answer area: clipboard scrub clears it; an incident is posted on the first occurrence and throttled afterwards.
- ☐ Right-click in the exam content area produces no menu and posts an incident (throttled).
- ☐ After submit, all three are restored to normal: chrome.exe runs freely, clipboard persists, right-click works.

Recommended model: Claude Opus 4.7 (alternative: Claude Sonnet 4.5).

Prompt for Windsurf:

Four modules under `client/app/lockdown/`.
`fullscreen.py`: class `FullscreenSubsystem`. On start: take the exam window's HWND (via `wininfo_id()` from Tkinter), set window styles via `SetWindowLongPtr` to remove `WS_CAPTION` / `WS_THICKFRAME` / `WS_SYSMENU` / `WS_MINIMIZEBOX` / `WS_MAXIMIZEBOX`, set `WS_EX_TOPMOST` via `SetWindowPos` with `HWND_TOPMOST`. Resize to cover the entire primary monitor including the taskbar area (use `GetSystemMetrics SM_CXSCREEN` / `SM_CYSCREEN`). Hide the taskbar separately via `FindWindowW` for "Shell_TrayWnd" then `ShowWindow` with `SW_HIDE` — and also hide the secondary "Start" button window (`FindWindowW` "Button" with title "Start") which on Windows 11 is its own HWND. Save all original window state (style flags, position/size, taskbar visibility) for restoration on stop.
A re-engage thread runs while `shutdown_event` is not set, polling every 1 second whether the exam window is still topmost and full-screen-sized. If not, re-apply. If the user manages to switch desktops (Win+Tab via mouse — though the mouse boundary subsystem makes that hard), the focus monitor will catch it. Post a `FULLSCREEN_BREACH` incident if a re-engage was needed (throttled to once per 5 seconds).
On stop: restore all saved window state, `ShowWindow` `SW_SHOW` the taskbar/Start button. Critical: if stop fails partway, ensure taskbar visibility is restored — wrap that step in its own try and never skip it.
`focus_monitor.py`: class `FocusMonitorSubsystem`. Background thread polling `GetForegroundWindow` every 500ms while `shutdown_event` is not set. Compares to the

exam window's HWND. On mismatch: post FOCUS_LOST incident with severity warning, description naming the foreground window's title (GetWindowTextW). Then call SetForegroundWindow on the exam HWND to yank focus back. SetForegroundWindow has annoying restrictions in modern Windows — work around with the AllowSetForegroundWindow + AttachThreadInput technique if needed, document in comments. Throttle to one incident per 2 seconds.

`mouse_boundary.py`: class `MouseBoundarySubsystem`. On start: call ClipCursor with the exam window's screen rectangle. Background thread re-applies ClipCursor every 500ms (Windows can release the clip on certain UI events) while shutdown_event is not set. On stop: ClipCursor with NULL to release. Post MOUSE_ESCAPE incident if at any poll the cursor is detected outside the rect (it shouldn't be possible normally, but if a DPI change or monitor change has occurred, it can briefly happen). Throttle.

`multi_monitor.py`: class `MultiMonitorSubsystem`. On start: count monitors via `screeninfo.get_monitors()` (or `EnumDisplayMonitors` via ctypes if screeninfo is fragile). If more than one, post MULTI_MONITOR_DETECTED incident with severity critical and call back to the manager via a `request_abort` callback (extend the manager from Part 23 to expose this — it will signal the exam screen to immediately submit-with-incident-and-return-to-dashboard treating this as a hard stop). Background thread re-checks every 3 seconds; if a new monitor is hot-plugged mid-exam, same outcome.

Update `LockdownManager` (Part 23) to expose `request_abort(reason: str)` that triggers a callback registered by the exam taking screen at start time. The exam screen's handler: stop the timer thread, force-submit the session (or call abort if pre_check), call stop on the manager, and navigate to dashboard with a toast explaining the reason.

Cross-version Windows compatibility: SetWindowLongPtr, SetWindowPos, ShowWindow, GetForegroundWindow, ClipCursor, FindWindowW are all stable across Windows 10 and 11. The Windows 11 Start button HWND is different from Windows 10's; handle both by trying multiple FindWindowW patterns and tolerating null (Windows 10 doesn't have the separate Start button HWND).

Commit as "security(lockdown): fullscreen, focus monitor, mouse boundary, multi-monitor".

Definition of Done: Exam goes fullscreen with the taskbar hidden. Alt+Tab attempt: focus monitor yanks back, FOCUS_LOST incident posted. Cursor cannot leave the exam window region. Plugging in a second monitor mid-exam: incident posted, exam force-aborts with a clear toast. After exam, taskbar reappears, window restores normal chrome, cursor moves freely.

Part 27 — Lockdown: Final Integration, Log Shipping Reliability, Wiring to Exam Screen

Stage: Security Goal: Bring all subsystems together under the manager, finalize the offline incident queue and bulk flush behavior, ensure clean shutdown under every exit path with stress testing, and confirm the full demo flow from login through locked exam through submit.

Prerequisites: Part 26.

Verification of Part 26.

- ☐ Exam goes truly fullscreen — taskbar hidden, no titlebar, covers second monitor area on primary screen at native resolution.
- ☐ Pressing Win key (already blocked by keyboard subsystem) and trying to move cursor to taskbar location — both fail.
- ☐ Forced focus to another window via `xdotool` / PowerShell `[Microsoft.VisualBasic.Interaction]::AppActivate` — focus monitor yanks back within 500ms with `FOCUS_LOST` incident.
- ☐ Plugging in second monitor mid-exam: `MULTI_MONITOR_DETECTED` critical incident; force-abort flow runs; user lands back on dashboard with a toast.
- ☐ After exam, all visual state is restored — no orphan topmost flags, taskbar back, cursor unconfined.

Recommended model: Claude Opus 4.7 (alternative: Claude Sonnet 4.5).

Prompt for Windsurf:

Finalize the lockdown integration.

Update `client/app/lockdown/manager.py` to register all subsystems in a fixed order at construction: `KeyboardLockdown`, `ProcessKillSubsystem`, `ClipboardScrubSubsystem`, `RightClickSuppressSubsystem`, `MultiMonitorSubsystem`, `FullscreenSubsystem`, `FocusMonitorSubsystem`, `MouseBoundarySubsystem`. Order matters for startup (multi-monitor first so we abort before going fullscreen) and reverse-order matters for shutdown (mouse boundary released before fullscreen reverts, etc.).

Reliability of incident shipping. Move the offline queue logic from Part 17's exam-taking screen into a new module `client/app/services/incident_pipeline.py`. The pipeline owns: an in-memory deque of queued incidents, a background flusher thread that every 5 seconds attempts a bulk POST to `/sessions/<id>/incidents`, retry with exponential backoff (1s, 2s, 4s, 8s capped at 30s) on transport failure, and a synchronous `flush_now()` method for use during submit. The pipeline handles the case where the bulk POST returns 409 because the session is already in a terminal state (it sometimes will, in submit/abort race conditions) by logging and discarding the batch — those incidents have value only while the session was active, and the server's automatic incidents (`LOCKDOWN_DISENGAGED`, etc.) will still be posted via direct (non-queued) calls in the abort/submit path that already succeed before terminal transition.

The lockdown manager and the exam screen both write to this pipeline. Initialize the pipeline once per session in the exam taking screen and pass a reference into the manager. Submit/abort path coordination: on submit, call `pipeline.flush_now()` synchronously, then call `manager.stop()`, then call POST submit. On abort (multi-monitor or any other request_abort), call `pipeline.flush_now()`, then `manager.stop()`, then POST abort. On window close (X button) mid-exam, intercept `WM_DELETE_WINDOW`: prompt confirmation, on confirm call abort path with reason=user. On unhandled exception in the exam screen: the manager's excepthook (Part 23) runs, calls `manager.stop()`, then `pipeline.flush_now()`. Unmount logs `LOCKDOWN_DISENGAGED`.

Stress test additions: write a small dev tool at `client/scripts/lockdown_smoke.py` that simulates a 10-second exam with all subsystems active and verifies that after the simulated submit, all subsystems report `is_started=false`, taskbar is visible, no topmost flag, ClipCursor is released. The script prints PASS/FAIL per subsystem.

Update the exam taking screen's `start_lockdown / stop_lockdown` to use the finalized manager. Also: explicitly handle the `SKIP_VM_CHECK` path — when dev mode is on, lockdown still engages (we want to test it); but when developing on non-Windows or when ADMIN privileges are missing entirely, allow a `SKIP_LOCKDOWN` env var (default false) for dev iteration. Document in `client/.env.example`.

Add a final docs file: `docs/LOCKDOWN_RUNBOOK.md` for the future you to consult after the demo, listing every subsystem, what it does, what incident types it produces, and how to disable each one via env vars (none of which should be on for the actual demo).

Cross-version Windows compatibility: confirm subsystems work on a Windows 10 VM and a Windows 11 host. Document any divergence in the runbook.

Commit as "security(lockdown): full integration, incident pipeline reliability, stress smoke test".

Definition of Done: End-to-end demo runs cleanly: student logs in, picks active exam, both VM gates pass, exam goes fullscreen-locked, all keyboard/process/clipboard/right-click/focus/cursor/multi-monitor defenses are active, student takes the exam normally, submit cleanly disengages everything. Force-killing the dev console mid-exam still restores the taskbar (excepthook works). Smoke test script passes all subsystems. The teacher's session detail shows `LOCKDOWN_ENGAGED` → many violation incidents (if the demoer tried things) → `LOCKDOWN_DISENGAGED` in chronological order.

Part 28 — PyInstaller Build: Spec, Hidden Imports, Admin Manifest, Single-File EXE

Stage: Packaging Goal: Produce a working single-file ExamSentinel.exe with admin elevation, custom icon, and all hidden imports correctly bundled. **Prerequisites:** Part 27.

Verification of Part 27.

- ☐ Smoke test script `client/scripts/lockdown_smoke.py` outputs PASS for every subsystem.
- ☐ Real demo run: student takes a full exam under lockdown without any visual glitch; submission works; teacher view shows the full incident timeline in correct order.
- ☐ X-button-close mid-exam: confirmation dialog → confirm → session aborted, dashboard shows it as `aborted_by_user`.
- ☐ Force-killing the dev process mid-exam (Task Manager → End Task on python.exe) — taskbar still returns when the next session starts, because next-launch lockdown reinitializes correctly even if previous exited dirtily.
- ☐ All env-var dev flags (`SKIP_VM_CHECK`, `SKIP_LOCKDOWN`) documented in `client/.env.example`.

Recommended model: Claude Opus 4.7 (alternative: Claude Sonnet 4.5). Opus — PyInstaller hidden imports for pywin32+wmi are an arcane art.

Prompt for Windsurf:

Produce the build pipeline for the Windows .exe.

Create `client/build/icon.ico` as a placeholder if missing — you can generate a simple icon programmatically (a square with a stylized lock symbol on the project's accent color background). 256x256 baseline; multiple resolutions inside the ICO.

Create `client/build/examsentinel.manifest` requesting administrator privileges with a level of "requireAdministrator" and uiAccess "false". Include the standard Windows compatibility blocks declaring support for Windows 10 and Windows 11 GUIDs so the app reports its target OS correctly.

Create `client/build/ExamSentinel.spec` (PyInstaller spec) configured for: single-file exe, windowed mode (no console), entry point `client/app/main.py`, name `ExamSentinel`, icon path, manifest path, working directory cleaned. Hidden imports must include — at minimum — every pywin32 submodule that is dynamically imported (win32api, win32con, win32gui, win32process, win32clipboard, win32event, pywintypes), every wmi module dependency, psutil internals (specifically psutil._psutil_windows which PyInstaller often misses), screeninfo, requests internal modules sometimes missed (charset_normalizer, certifi). Include `tkinter` explicitly. Also bundle the Tcl/Tk runtime (PyInstaller's default behavior, but verify in the spec).

Datas: include any text resources used at runtime — the dummy icon, the manifest (already embedded but include for symmetry), and any JSON configs the app reads. There shouldn't be many — keep this list minimal.

Create `client/build/build.bat` script that:

1. Activates the venv at `client/.venv`.
2. Installs/updates `client/requirements.txt`.
3. Runs `pyinstaller --clean ExamSentinel.spec` from `client/build/`.
4. After success, copies the output `dist/ExamSentinel.exe` to `client/build/release/ExamSentinel.exe` with the current ISO date appended in the filename.
5. Prints next-step instructions: where the exe is, that it requires admin (will trigger UAC), that examiners need internet to reach the Railway backend, and that the `API_BASE_URL` it was built with came from `client/.env`.

Important PyInstaller advice the prompt should include verbatim: build artifacts under `client/build/` (`build/` folder produced by PyInstaller and `dist/` folder) must be gitignored — confirm this in `.gitignore` from Part 4 covers them; if not, append. Antivirus engines often false-positive single-file PyInstaller binaries — document in `docs/DEMO_DAY.md` (Part 30) the right-click → Properties → Unblock workaround on examiner machines, plus the option to add a Defender exclusion for the demo folder.

Cross-version Windows compatibility:

- Ensure cross-version Windows compatibility (Windows 10 and 11)

- Avoid OS-specific libraries or Windows-version-specific APIs beyond what the security engine necessarily uses
- Package using PyInstaller for standalone Windows executable

Run a first build. Inspect the output for missing-module warnings. For each, add a `--hidden-import` to the spec or a `hiddenimports` entry. Re-run until clean.

Commit the spec, manifest, batch file, and any newly-added hidden imports as "packaging: PyInstaller spec, admin manifest, build script".

Definition of Done: Running `client/build/build.bat` produces `client/build/dist/ExamSentinel.exe`. Double-clicking the exe triggers UAC; on consent, the splash appears, server health check runs against the configured `API_BASE_URL`, login proceeds. The full happy path (login → start exam → integrity checks → exam under lockdown → submit → result) runs from the .exe identically to running from source. The exe is a single file (~50-80 MB is normal); copying it to a freshly imaged Windows 11 machine and running it (with internet to Railway) succeeds without installing Python.

Part 29 — Smoke Test on a Clean Windows 11 Machine

Stage: Packaging **Goal:** Run the full demo flow on a clean Windows 11 environment (a freshly created VM is fine — the irony is intentional, but for this test you'll DISABLE VM detection via `SKIP_VM_CHECK` to verify everything else works), find any issues, fix them, and produce a tested release-ready exe. **Prerequisites:** Part 28.

Verification of Part 28.

- ☐ `build.bat` succeeds with zero PyInstaller warnings about missing modules.
- ☐ `dist/ExamSentinel.exe` size is between 40 and 100 MB.
- ☐ Double-clicking the exe triggers UAC.
- ☐ On a UAC consent, the splash appears; on UAC decline, the app exits cleanly without crashing.
- ☐ The `release/` subfolder contains a copy with the date-stamped filename.
- ☐ The .exe runs the integrity checks → exam → submit flow against the Railway backend on the build machine.

Recommended model: Claude Sonnet 4.5 (alternative: Claude Opus 4.7).

Prompt for Windsurf:

Run a structured smoke test on a clean Windows 11 environment that does not have Python installed. Use a freshly imaged Windows 11 VM if available, or a colleague's untouched laptop. The intent is to catch: missing runtime dependencies that PyInstaller didn't bundle, antivirus false-positives, UAC behavior on a non-developer machine, network/CORS issues against Railway, layout glitches at default 100% and 125% DPI, taskbar restore after lockdown.

Ensure `SKIP_VM_CHECK` is set to true on the test machine for this run only — if testing inside a VM, the VM gates would otherwise block the test of the rest of the system. Document this in the runbook clearly: `SKIP_VM_CHECK` is a developer flag and is OFF for the actual demo.

Smoke test flow:

1. Copy `ExamSentinel.exe` to the clean machine. If antivirus quarantines, document the workaround (right-click → Unblock; Defender exclusion).
2. Set `API_BASE_URL` to the Railway URL via the appropriate mechanism — since the `.env` file isn't bundled into the exe by default, document that the production API URL must be baked in at build time. Update `client/app/config.py` to fall back to a build-time-injected default if no env or local `.env` is found. Set this default to the Railway URL before final build.
3. Run the `.exe`; pass through UAC.
4. Splash → server reachable → login screen.
5. Register a new student through the UI (use the seed teacher's existing course's enrollment flow first via the seed teacher logged in on the dev machine, or — better — extend the seed to include test-suitable students with known credentials). Or just use seed credentials.
6. Login as student → see active exam → click Start Exam.
7. Integrity check: standard VM "skipped" with `SKIP_VM_CHECK` on; stealth same. Continue.
8. Exam-taking screen goes fullscreen; lockdown engages. Try Alt+Tab, Win key, right-click, copy/paste, launching Chrome — every attempt fails or is killed; incidents post.
9. Submit. Lockdown disengages cleanly — taskbar returns, cursor is free, can Alt+Tab.
10. Login as teacher on a second machine (or same machine after logout), open Reports, find the just-submitted session, see the full incident timeline.
11. Test on Windows 10: same flow on a Windows 10 machine if available — confirm subsystems behave identically.

File any issues found as bullet items in `docs/SMOKE_TEST_LOG.md` with reproduction steps, root cause, and fix. Apply fixes and rebuild. Re-run the smoke test until clean.

Common fix categories to anticipate: missing pywin32 post-install step (PyInstaller bundles dlls but not the post-install scripts — document workaround); antivirus false-positive on the unsigned binary (document right-click Unblock); Windows 10 lacking the separate Start button HWND (Part 26 handled this — verify); high-DPI font scaling issues in Tkinter (set `tk.call('tk', 'scaling', ...)` early in main).

Cross-version Windows compatibility:

- Ensure cross-version Windows compatibility
- Avoid OS-specific libraries or Windows-version-specific APIs (with the documented exceptions for the security engine)

- Final binary remains a standalone Windows executable produced by PyInstaller

Commit smoke test log and any fix commits as "qa: smoke test pass on clean Windows 11/10".

Definition of Done: A pristine Windows 11 machine without Python, given only the .exe, runs the full happy demo flow against the live Railway backend with all lockdown defenses working, and the teacher view on a second machine reflects everything. `docs/SMOKE_TEST_LOG.md` exists with at least the run report (issues empty if perfect, otherwise issues with their fixes documented).

Part 30 — Demo Day Checklist, README, Examiner-Facing Docs

Stage: Packaging **Goal:** Produce the user-facing project artifacts: top-level README, demo day step-by-step walkthrough for the examiners, FAQ for likely examiner questions, and a final tag/release on GitHub. **Prerequisites:** Part 29.

Verification of Part 29.

- ☐ Smoke test log exists and reports a clean run on at least one Windows 11 machine without Python pre-installed.
- ☐ All lockdown defenses verified visible on the clean machine (Alt+Tab blocked, taskbar hidden, Chrome killed, clipboard scrubbed, right-click suppressed).
- ☐ Submit cleanly restores environment.
- ☐ Teacher view on a second machine shows the full incident timeline for the just-completed test run.
- ☐ If Windows 10 was tested, all subsystems behaved identically.

Recommended model: Claude Sonnet 4.5 (alternative: Claude Opus 4.7).

Prompt for Windsurf:

Produce three documents and a final release.

Top-level `README.md`: rewrite from the placeholder created in Part 4 to be an actual project overview. Sections: project name and one-line description; a screenshot block (placeholder paths under `docs/screenshots/` — the human will fill these); features list (lockdown features, VM detection, real-time incident logging, teacher review); tech stack; project structure summary linking into `docs/ARCHITECTURE.md`; getting started for developers (clone, server setup with venv and `.env`, client setup, running locally, deploying to Railway, building the .exe); demo flow summary linking into `docs/DEMO_DAY.md`; acknowledgments (PUCIT FYP 2026); license placeholder.

`docs/DEMO_DAY.md`: a literal step-by-step the human walks through with examiners.

Sections:

Pre-demo setup (the night before): build a fresh .exe with `API_BASE_URL` pointing to Railway; copy it to USB or upload to a download link; verify Railway is up; verify seed data is present (run `/api/_seed` if not); have credentials handy.

Live walkthrough script. Step 1: install/run the .exe on the examiner's machine; trigger UAC. Step 2: register or login as teacher; show course list; open one course; show student list; click

Create Exam; demonstrate adding MCQs and short-answer questions; activate. Step 3: register or login as student on a second machine (or after logging the teacher out); show available exams; click Start. Step 4: integrity check screen runs both VM gates; on bare metal both pass; mention briefly what's happening per check. Step 5: exam goes fullscreen; demonstrate lockdown by attempting cheating: Alt+Tab does nothing, Win key does nothing, opening Chrome via Win+R is impossible (Win blocked), opening from Start menu — Start menu also blocked; if examiner brings their own Chrome already running, watch it die in the process scanner; copy/paste cleared in the clipboard scrubber; right-click suppressed; if they plug in a second monitor, exam aborts. Step 6: student answers a question, submits. Step 7: switch to teacher view; show the incident log with timestamps and forensic fields; show manual short-answer grading. Step 8: simulate a VM attempt: run the .exe inside a VirtualBox guest; show standard VM detection blocking; close and reopen — it blocks again; show the teacher's view of the aborted_vm sessions.

Likely examiner questions and prepared answers:

- "Can this be bypassed?" — Yes, any software-only solution can be bypassed by a determined adversary with hardware access; we raise the cost and produce evidence; defense in depth not impenetrable.
- "Why no Windows 7 / Linux support?" — Out of scope; security engine relies on Windows 10/11 APIs; pywin32 calls are not portable; supporting other platforms would require redesigning the lockdown layer (e.g., X11 vs Wayland on Linux behaves differently).
- "Why software-only and not hardware proctoring (e.g., webcam stream)?" — We considered it; webcam proctoring requires a remote proctor or a separate AI pipeline, both out of scope and ethically heavier (privacy implications); incident-log approach is reviewable, auditable, and doesn't surveil the student's room.
- "What's the research contribution?" — The stealth-VM detection layer combining CPUID, RDTSC timing, sleep skew, deep registry artifacts, and thermal-zone heuristics catches hardened VMs that survive standard detection; the offline-queued incident pipeline guarantees evidence delivery despite intermittent connectivity; the joined-table-inheritance domain model cleanly accommodates role-specific fields without polluting the User table.
- "Why Railway not AWS/GCP/Azure?" — Cost (\$5/month vs \$5+ for the smallest cloud VPS plus management overhead), zero-ops Postgres, GitHub-driven deploys, sufficient for FYP demo scale.
- "What's the database schema?" — Refer to ER diagram in `docs/ARCHITECTURE.md`; ten core entities, joined-table inheritance for User → Student/Teacher, junction table for course enrollments.

Troubleshooting on demo day: server unreachable → check Railway status page, restart the service, confirm CORS origins; antivirus quarantine → right-click Unblock or temporary Defender exclusion; UAC declined → re-launch and accept; VM gate falsely failing on bare

metal → check `docs/SMOKE_TEST_LOG.md` for environment-specific notes (some Dell laptops have weird BIOS strings; document any encountered).

`docs/CHANGELOG.md`: list versions starting with 1.0.0 today, summarizing what shipped.

Final actions:

- Update `README.md` with version 1.0.0.
- Tag the commit as `v1.0.0` (`git tag -a v1.0.0 -m "..."`).
- Push the tag to GitHub.
- Create a GitHub Release from the tag, attaching the latest dated .exe from `client/build/release/` as a binary asset (the human does this in the GitHub UI; the prompt instructs how).

Cross-version Windows compatibility: documented as a feature, not a bug — the .exe runs on Windows 10 and 11.

Commit as "release: v1.0.0 — README, demo day docs, FAQ, changelog".

Definition of Done: The repository has a polished README pointing at all the right places.

`docs/DEMO_DAY.md` is a script the human can literally read off during the FYP defense. The FAQ section covers the questions examiners are most likely to ask, with prepared answers. The v1.0.0 git tag is pushed; a GitHub Release exists with the .exe attached. The human is ready to demo.

Closing Notes

You've built ExamSentinel. The system at v1.0.0 covers the locked-down exam loop end-to-end with both VM gates, full Windows lockdown, real-time forensic logging, teacher review and analytics, and a packaged Windows binary. None of this is bypass-proof — no software-only system is — but every defense produces an evidence trail the teacher can review, which is the realistic security posture for a demo-grade FYP.

If you're tight on time during the build, the optional features that can be cut without breaking the demo are: per-question reordering in the exam editor (Part 16), the analytics drawer (Part 18), the audit-log clipboard helper (Part 22), the smoke test script (Part 27 — just verify manually), and the date-stamped release copy (Part 28 — the plain dist/ExamSentinel.exe is fine). Do not cut any of: the architecture and security-design documents (Parts 1-3), the seed endpoint (Part 11), the integrity-check screen (Parts 17, 20, 22), the keyboard hook (Part 24), the fullscreen + focus monitor (Part 26), the manager's guaranteed teardown (Parts 23, 27), the PyInstaller admin manifest (Part 28).

Demo day: stay calm, walk the examiners through `docs/DEMO_DAY.md`, breathe between steps, and remember the defense is about showing the system works as designed — not about it being unbreakable.